

✓ CHAPTER 6 COMBINATIONAL LOGIC DESIGN

6.1 Introduction

Digital logic circuits may be combinational or sequential. A *combinational* circuit consists of logic gates whose outputs at any time are determined directly from the present combination of inputs without regard to previous inputs. A combinational circuit performs a specific information processing operations fully specified logically by a set of Boolean functions. *Sequential* circuits employ memory element in addition to logic gates. Their outputs are a function of the inputs and the state of the memory elements. The state of memory elements, in turn, is a function of previous inputs. As a consequence, the outputs of a sequential circuit depend not only on present inputs, but also on past inputs, and the circuit behavior must be specified by a time sequence of inputs and internal states. Sequential circuits are discussed in Chapter 10.

The purpose of this chapter is to use the knowledge acquired in previous chapters and formulate systematic design and analysis procedures of combinational circuits.

A combinational circuit consists of input variables, logic gates, and output variables. The logic gates accept signals from the inputs and generate signals to the outputs. This process transforms binary information from the given input data to the required output data. Obviously, both input and output are represented by binary signals, i.e., they exist in two possible values, one representing 1 and the other 0. A block diagram of a combinational circuit is shown in Figure 6.1. The n input binary variables come from an external source; the m output variables go to an external destination. In many applications, the source and/or destination are storage registers (discussed in Chapter 11) located either in the vicinity of the combinational circuit or in a remote external device.

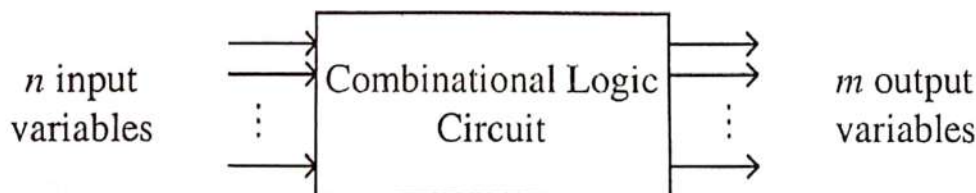


Figure 6.1 Block diagram of a combinational circuit.

For n input variables, there are 2^n possible combinations of binary input values. For each possible input combination, there is one and only one possible output combination. A combinational circuit can be described by m Boolean functions, one for each output variable. Each output function is expressed in terms of the n input variables.

6.2 Design of Combinational Circuits

The design of combinational circuits involves the following steps:

1. The problem is stated.

2. The number of available input variables and required output variables is determined. The input and output variables are assigned letter symbols and logic values.
3. The truth table that defines the required relationships between inputs and outputs is derived.
4. The simplified Boolean function for each output is obtained.
5. The logic diagram is obtained.

A truth table for a combinational circuit consists of input columns and output columns. The binary values for the outputs are determined from examination of the stated problem. The output functions specified in the truth table give the exact definition of the combinational circuit. It is important that the verbal specifications be interpreted correctly into a truth table. Sometimes the designer must use his intuition and experience to arrive at the correct interpretation. Word specifications are very seldom complete and exact. Any wrong interpretation that results in an incorrect truth table produces a combinational circuit that will not fulfil the stated requirements.

The output Boolean functions from the truth table are simplified by any available method, such as algebraic simplification, the K-map method, the Quine-McCluskey method, or the iterated consensus method. Usually there will be a variety of simplified expressions from which to choose. However, in any particular application, certain restrictions, limitations, and criteria will serve as a guide in the process of choosing a particular algebraic expression. A practical design method would have to consider such conditions as

- (1) minimum number of gates,
- (2) minimum number of inputs to a gate,
- (3) minimum propagation time of the signal through the circuit,
- (4) minimum number of interconnections, and
- (5) limitations of the driving capabilities of each gate.

Since all these criteria can not be satisfied simultaneously, and since the importance of each constraint is dictated by the particular application, it is difficult to make a general statement as to what constitutes an acceptable simplification. In most cases the simplification begins by satisfying an elementary objectives, such as producing a simplified Boolean function in a standard form, and from that proceeds to meet any other performance criteria.

In practice, designers tend to go from the Boolean functions to a wiring list that shows the interconnections among various standard logic gates. In that cases the design need not go any further than the required simplified output Boolean functions. However, a logic diagram is helpful for visualizing the gate implementation of the expressions.

The design procedure is demonstrated using the following example.

1. **The problem statement:** The door of an explosive-warehouse is operated on by four persons, namely, *A*, *B*, *C*, and *D*. Person *A* along with at least any one from *B*, *C*, and *D* can open the door. If *A* is absent, *B*, *C*, and *D* can open the

door altogether. A combinational logic circuit is to be designed for controlling the door.

- Four input lines are required for four keys operated by four persons. The four input variables are assigned letter symbols A , B , C , and D after the names of the four persons. The input variables are assigned the following logic values:

The key is operated for opening the door: 1

The key is operated for closing the door: 0

One output line is required for controlling the lock of the door. The output variable is assigned the letter symbol X and the following logic values:

The lock is opened: 1

The lock is closed: 0

- The truth table defining the relationship between inputs and output is derived and shown in Table 6.1.

Table 6.1 Truth table for explosive-warehouse door control example.

A	B	C	D	X
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

- The simplified Boolean function for the truth table of Table 6.1 is obtained by K-map method in Figure 6.2, which gives

$$X = AB + AC + AD + BCD$$

		CD			
		00	01	11	10
AB	00	0	0	0	0
	01	0	0	1	0
	11	1	1	1	1
	10	0	1	1	1

Figure 6.2 Simplification of the function of Table 6.1 by K-map.

5. The logic diagram of the door control circuit is shown in Figure 6.3.

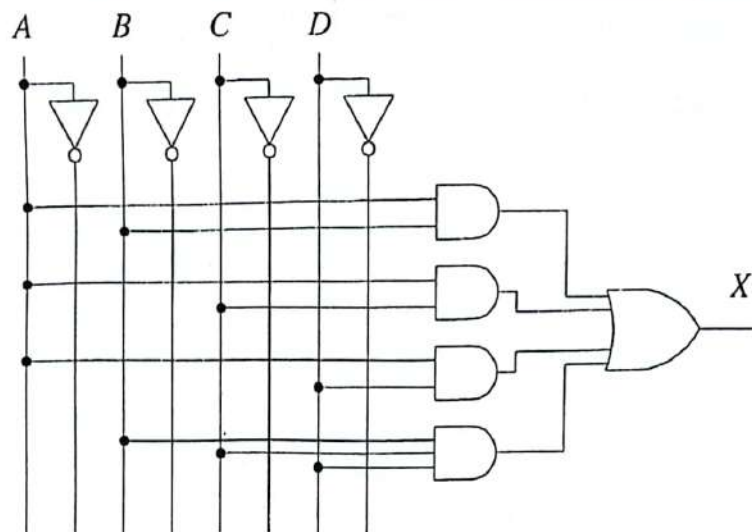


Figure 6.3 Logic diagram for the explosive-warehouse door control example.

The design of some typical elementary functions important for the understanding of digital computers and systems will be discussed in the following chapters.

6.3 Analysis of Combinational Circuits

The design of a combinational circuit starts from the verbal specifications of a required function and culminates with a logic diagram. The analysis of a combinational circuit is somewhat the reverse process. It starts with a given logic diagram and culminates with a set of output Boolean functions or a truth table or a verbal explanation of the circuit operation. If the logic diagram to be analyzed is accompanied by a function name or an explanation of what it is assumed to accomplish, then the analysis problem reduces to a verification of the stated function.

The first step in the analysis is to make sure that the given circuit is combinational and not sequential. The diagram of a combinational circuit has logic gates with no feedback paths or memory elements. A feedback path is a connection from the output of one gate to the input of a second gate that forms part of the input to the first gate. Feedback paths or memory elements in a digital circuit define a sequential circuit and must be analyzed according to procedures outlined in Chapter 10.

Derivation of Boolean Expressions

Once the logic diagram is verified as a combinational circuit, the Boolean expressions can be derived using the following steps:

1. Label with arbitrary symbols all gate outputs that are a function of the external input variables only. Obtain the Boolean function for each gate.
2. Label with other arbitrary symbols those gates that are a function of input variables and/or previously labeled gates. Find the Boolean functions for these gates. Repeat the above process until the outputs of the circuit are obtained.

- By repeated substitution of previously defined functions, obtain the output Boolean functions in terms of external input variables only.

The procedure is demonstrated below for the example combinational circuit of Figure 6.4.

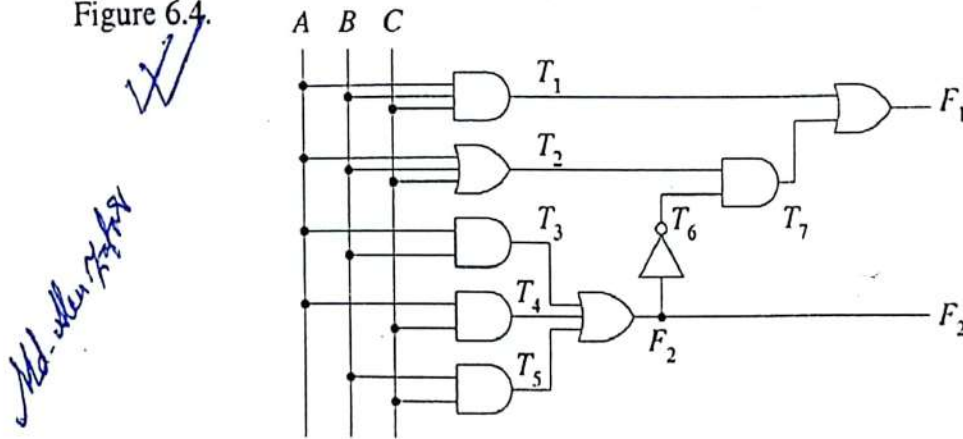


Figure 6.4 Combinational logic circuit for analysis example.

- The outputs of gates that are a function of external input variables only are T_1 , T_2 , T_3 , T_4 , and T_5 . The Boolean functions for these five outputs are

$$T_1 = ABC$$

$$T_2 = A + B + C$$

$$T_3 = AB$$

$$T_4 = AC$$

$$T_5 = BC$$

- Now we consider outputs of gates that are a function of already defined symbols:

$$F_2 = T_3 + T_4 + T_5$$

$$T_6 = F_2'$$

$$T_7 = T_2 T_6$$

$$F_1 = T_1 + T_7$$

- The output Boolean function F_2 in terms of external input variables only is obtained as follows:

$$F_2 = T_3 + T_4 + T_5 = AB + AC + BC$$

The output Boolean function F_1 in terms of external input variables only is obtained by repeated substitution of previously defined functions as follows:

$$\begin{aligned}
F_1 &= T_1 + T_7 \\
&= ABC + T_2 T_6 \\
&= ABC + (A + B + C)F_2' \\
&= ABC + (A + B + C)(AB + AC + BC)' \\
&= ABC + (A + B + C)(A' + B')(A' + C')(B' + C') \\
&= ABC + (AB' + A'B + A'C + B'C)(A'B' + A'C' + B'C' + C') \\
&= ABC + AB'C' + A'BC' + A'B'C
\end{aligned}$$

Derivation of Truth Table

Construction of truth table is straightforward from the derived Boolean functions. However, the truth table can be obtained directly from the logic diagram without going through the derivations of the Boolean functions by using the following steps:

1. Determine the number of input variables to the circuit. For n inputs, form the 2^n possible input combinations of 1s and 0s.
2. Label the outputs of gates with arbitrary symbols.
3. Obtain the truth table for the outputs of those gates that are a function of external input variables only.
4. Proceed to obtain the truth table for the outputs of those gates that are a function of previously defined values until the columns for all outputs are determined.

This process is demonstrated using the circuit of Figure 6.4 as follows:

1. The input variables of the circuit are A , B , and C . The $2^3 = 8$ input combinations of 1s and 0's for three variables are shown in the left column of Table 6.2.
2. The gates are labeled as in Figure 6.4.
3. Outputs T_1 , T_2 , T_3 , T_4 , and T_5 are functions of external input variables only and they are

$$T_1 = ABC$$

$$T_2 = A + B + C$$

$$T_3 = AB$$

$$T_4 = AC$$

$$T_5 = BC$$

Truth table for T_1 , T_2 , T_3 , T_4 , and T_5 are derived from input combinations and are shown in the Table 6.2.

4. Outputs F_2 , T_6 , T_7 , and F_1 are function of previously defined values and they are

$$F_2 = T_3 + T_4 + T_5$$

$$T_6 = F_2'$$

$$T_7 = T_2 T_6$$

$$F_1 = T_1 + T_7$$

Truth table for F_2 is derived from the truth table of T_3 , T_4 , and T_5 . Truth table for T_6 is derived from the truth table of F_2 . Truth table for T_7 is derived from the truth tables of T_2 and T_6 . Finally, truth table for F_1 is derived from the truth tables of T_1 and T_7 . These truth tables are shown in Table 6.2.

Table 6.2 Truth table for the operation of the logic diagram of Figure 6.4.

ABC	T_1	T_2	T_3	T_4	T_5	F_2	T_6	T_7	F_1
000	0	0	0	0	0	0	1	0	0
001	0	1	0	0	0	0	1	1	1
010	0	1	0	0	0	0	1	1	1
011	0	1	0	0	1	1	0	0	0
100	0	1	0	0	0	0	1	1	1
101	0	1	0	1	0	1	0	0	0
110	0	1	1	0	0	1	0	0	0
111	1	1	1	1	1	1	0	0	1

6.4 Two-Level NAND Circuits

Digital circuits are more frequently constructed with NAND and NOR gates than with AND and OR gates. NAND and NOR gates are easier to fabricate with electronic components and are the basic gates in all IC digital logic families. Because of the prominence of NAND and NOR gates in the design of digital circuits, rules and procedures have been developed for the conversion from Boolean functions given in terms of AND, OR, and NOT into equivalent NAND or NOR logic diagrams.

The implementation of a Boolean function with NAND gates requires that the function be simplified in the sum of products form. To see the relationship between a sum of products expression and its equivalent NAND implementation, consider the logic diagram drawn in Figure 6.5. All three diagrams are equivalent and implement the function

$$F = A + BC + CD$$

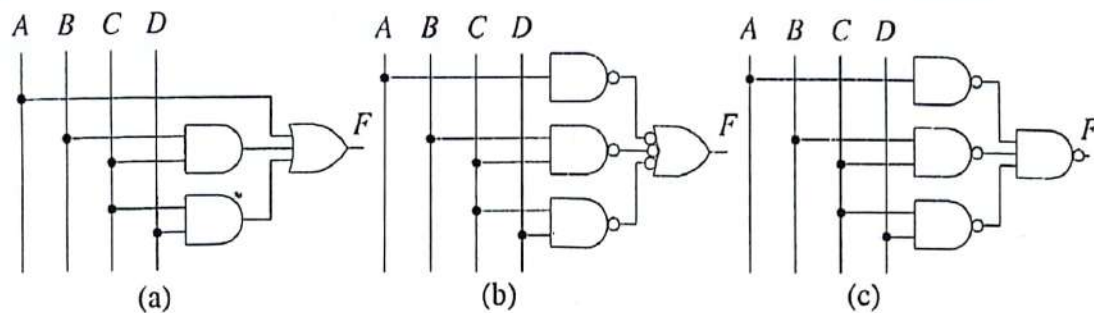


Figure 6.5 Three ways to implement $F = A + BC + CD$.

The function is implemented in Figure 6.5(a) in sum of products form with AND and OR gates. In Figure 6.5(b), the AND gates are replaced by NAND gates and the OR gate is replaced by a NAND gate with an invert-OR symbol (see Figure 4.8). The single variable A is complemented using one-input NAND gate (see Figure 4.8) and applied to the second-level invert-OR gate. Remember that a small circle denotes complementation. Therefore, two circles on the same line represents double complementation and both can be removed. The complement of A goes through a small circle which complements the variable again to produce the uncomplemented value of A . Removing the small circles in the gates of Figure 6.5(b) produces the circuit in Figure 6.5(a). Therefore, the two diagrams implement the same function and are equivalent. In Figure 6.5(c), the output NAND gate is redrawn with the conventional symbol. The diagram in Figure 6.5(c) is equivalent to the one in Figure 6.5(b), which in turn is equivalent to the diagram in Figure 6.5(a). Note the similarity between the diagrams in Figure 6.5(a) and 6.5(c). The AND and OR gates have been changed to NAND gates and an additional one-input NAND gate has been included with the single variable A .

The NAND implementation in Figure 6.5(c) can be verified algebraically using DeMorgan's theorem as follows:

$$F = (A'(BC)'(CD)')' = A + BC + CD$$

From the transformation shown in Figure 6.5, we conclude that a Boolean function can be implemented with two levels of NAND gates. The rule for obtaining the NAND logic diagram from a Boolean function is as follows:

1. Simplify the function and express it in sum of products form.
2. Draw a NAND gate for each product term of the function that has at least two literals. The inputs to each NAND gate are the literals of the product term. This constitutes a group of first-level gates. A term with a single literal requires an inverter in the first level. Therefore, complement a single literal and apply as an input to the second level.
3. Draw a single NAND gate in the second level, with inputs coming from the first level.

The procedure is demonstrated for the following function

$$F(A, B, C, D) = \sum(0, 1, 2, 4, 5, 6, 8, 10, 12, 14, 15)$$

Step 1. The simplification of the function as sum of products using K-map is shown in Figure 6.6, which gives

$$F(A, B, C, D) = D' + A'C' + ABC$$

CD \ AB	00	01	11	10
00	1	1		1
01	1	1		1
11	1		1	1
10	1			1

Figure 6.6 Simplification of $F(A, B, C, D) = \sum(0,1,2,4,5,6,8,10,12,14,15)$ in sum of products form using K-map.

Steps 2 and 3. The two-level NAND implementation of the simplified function using steps 2 and 3 is shown in Figure 6.7.

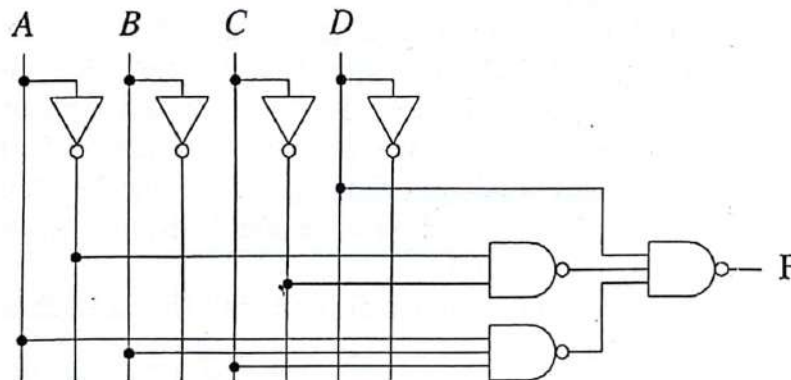


Figure 6.7 Two-level NAND implementation of $F(A, B, C, D) = D' + A'C' + ABC$.

6.5 Two-Level NOR Circuits

The NOR function is the dual of the NAND function. For this reason, all procedures and rules for NOR logic are the dual of the corresponding procedures and rules developed for NAND logic.

The implementation of a Boolean function with NOR gates requires that the function be simplified in product of sums form. A product of sums expression specifies a group of OR gates for the sum terms, followed by an AND gate to produce the product. The transformation from the OR-AND to the NOR-NOR diagram is depicted in Figure 6.8 for the product of sums expression

$$F = A(B + C)(C + D)$$

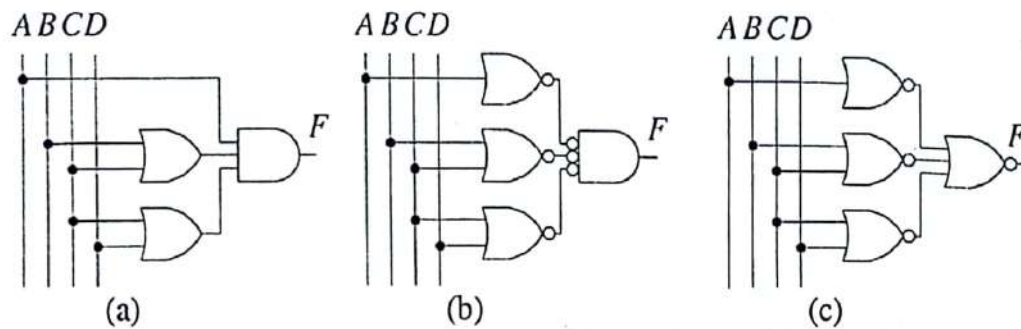


Figure 6.8 Three ways to implement $F = A(B + C)(C + D)$.

The rule for obtaining the NOR logic diagram from a Boolean function can be derived from this transformation and is as follows:

1. Simplify the function and express it in product of sums form.
2. Draw a NOR gate for each sum term of the function that has at least two literals. The inputs to each NOR gate are the literals of the sum term. This constitutes a group of first-level gates. A term with a single literal requires an inverter in the first level. Therefore, complement a single literal and apply as an input to the second level.
3. Draw a single NOR gate in the second level, with inputs coming from the first level.

The procedure is demonstrated for the function

$$F(A, B, C, D) = \prod(0, 1, 4, 5, 7, 8, 9, 10, 11, 12, 13)$$

Step 1. The simplification of the function as product of sums form using K-map is shown in Figure 6.9, which gives

$$F(A, B, C, D) = C(A' + B)(A + B' + D')$$

AB \ CD	CD			
	00	01	11	10
00	0	0		
01	0	0	0	
11	0	0		
10	0	0	0	0

Figure 6.9 Simplification of $F(A, B, C, D) = \prod(0, 1, 4, 5, 7, 8, 9, 10, 11, 12, 13)$ in product of sums form using K-map.

Steps 2 and 3. The two-level NOR implementation of the simplified function is shown in Figure 6.10.

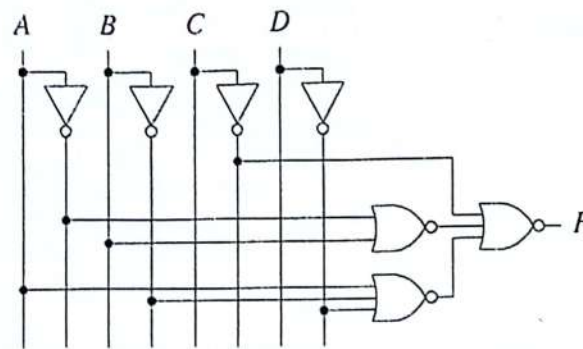


Figure 6.10 Two-level NOR implementation of $F(A, B, C, D) = C(A' + B)(A + B' + D')$.

6.6 Multi-Level NAND Circuits

6.6.1 Boolean Function Implementation with Multi-Level NAND Circuit

The implementation of Boolean functions with multi-level NAND circuits may be obtained by means of a simple block diagram manipulation technique. This method requires that two other logic diagrams be drawn prior to obtaining the NAND logic diagram. Nevertheless, the procedure is very simple and straightforward:

1. From the given algebraic expression, draw the logic diagram with AND, OR, and NOT gates. Assume that both the uncomplemented and complemented inputs are available.
2. Draw a second logic diagram with the equivalent NAND logic, as given in Figure 4.6, substituted for each AND, OR, and NOT gate.
3. Remove any two cascaded inverters from the diagram, since double inversion does not perform a logic function. Remove inverters connected to single external inputs and complement the corresponding input variable. The new logic diagram obtained is the required NAND gate implementation.

This procedure is demonstrated using the following examples.

Example 1: The function is $F = B(C + AD) + CD'$.

1. The AND-OR implementation of this function is shown in the logic diagram of Figure 6.11(a).
2. For each AND gate, we substitute a NAND gate followed by an inverter; for each OR gate, we substitute input inverters followed by a NAND gate. This substitution follows directly from the logic equivalence of Figure 4.6 and is shown in the diagram of Figure 6.11(b). This diagram has seven inverters (excluding the inverters used for indicating the complemented input) and five two-input NAND gates listed with numbers inside the gate symbol.
3. Pairs of inverters connected in cascade (from each AND box to each OR box) are removed since they form double inversion. The inverter connected to input C is removed and the input is connected to C' input. The result is the NAND logic diagram shown in Figure 6.11(c), with the number inside each symbol identifying the gate from Figure 6.11(b).

This example demonstrated that the number of NAND gates required to implement the Boolean function is equal to the number of AND-OR gates, provided both the uncomplemented and the complemented inputs are available.

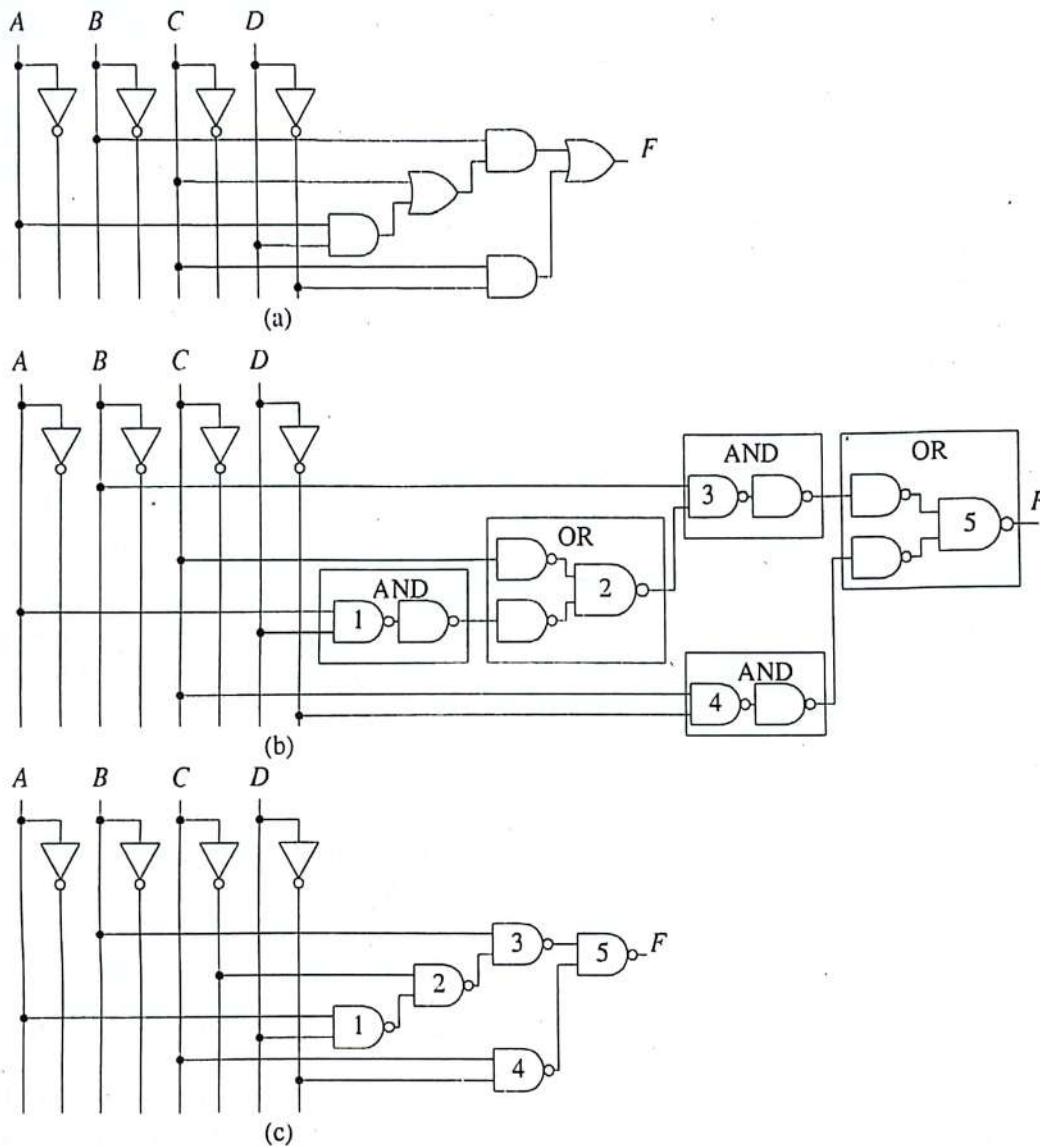


Figure 6.11 Implementation of $F = B(C + AD) + CD'$ with NAND gates.

Example 2: The function is $F = (A' + C)(BC + D)$.

1. The AND-OR implementation is shown in Figure 6.12(a).
2. The NAND logic substitution is shown in Figure 6.12(b).
3. One pair of cascaded inverters from gate 1 to gate 3 is removed. The three external inputs A' , C, and D, which directly go to inverters, are complemented and the corresponding inverters are removed. The final NAND gate implementation is shown in Figure 6.12(c).

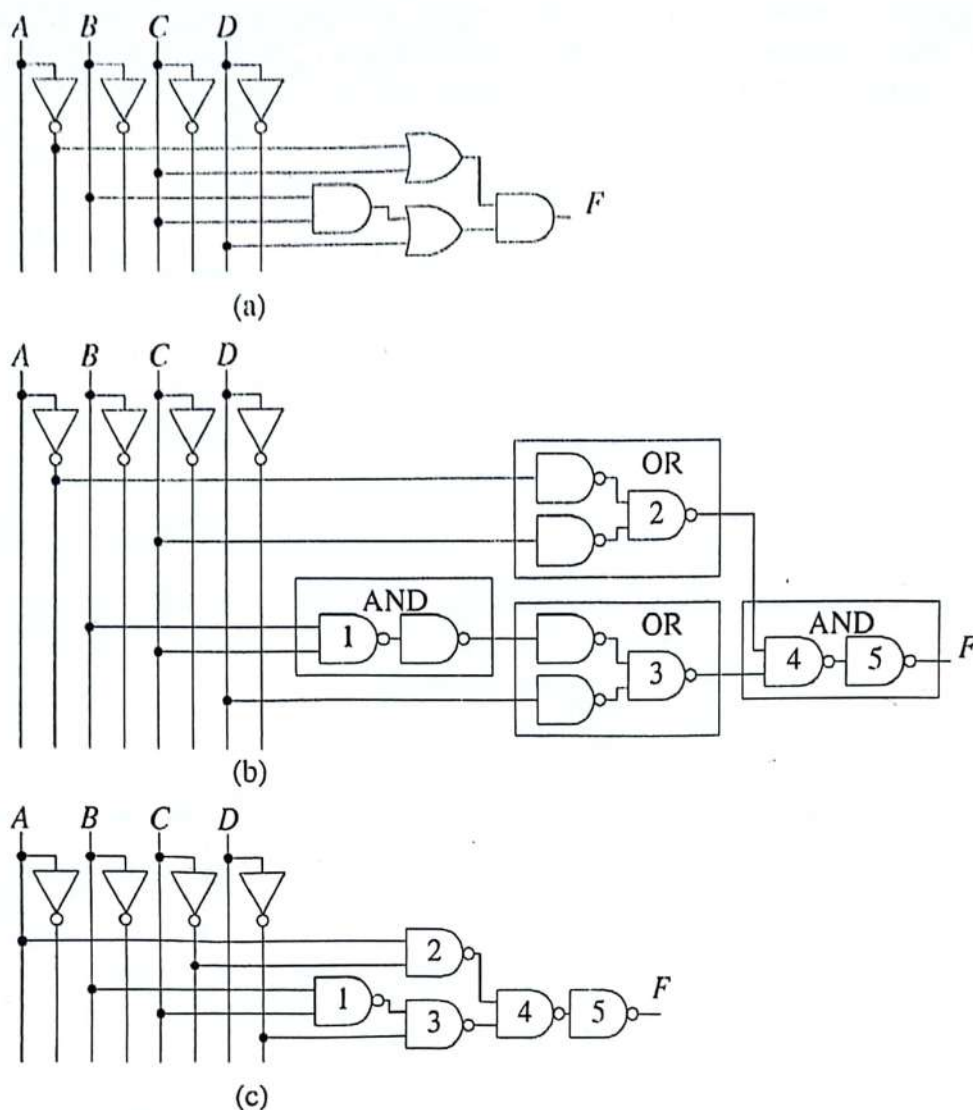


Figure 6.12 Implementation of $F = (A' + C)(BC + D)$ with NAND gates.

The number of NAND gates for this example is equal to the number of AND-OR gates plus an additional inverter in the output (NAND gate 5). In general, the number of NAND gates required to implement a function equals the number of AND-OR gates, except for an occasional inverter. This is true provided both the uncomplemented and the complemented inputs are available.

The block diagram method is somewhat tiresome to use because it requires the drawing of two logic diagrams to obtain the answer in a third.

6.6.2 Analysis of Multi-Level NAND Circuits

The foregoing procedure considered the problem of deriving a NAND logic diagram from a given Boolean function. The reverse process is the analysis problem, which starts with a given NAND logic diagram and culminates with a Boolean expression or a truth table. The analysis of NAND logic diagrams follows the same procedure presented in Section 6.3. The only difference is that NAND logic requires a repeated application of DeMorgan's theorem.

Derivation of the Boolean Expression by Algebraic Manipulation

The procedure for deriving the Boolean expression from a logic diagram is outlined in Section 6.3. This procedure is demonstrated for the NAND logic diagram of Figure 6.11(c), which is redrawn in Figure 6.13.

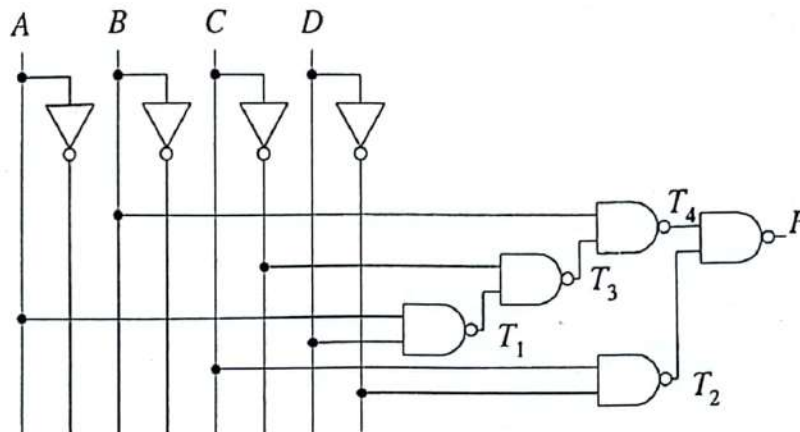


Figure 6.13 Analysis example of NAND circuit.

First, all gate outputs are labeled with arbitrary symbols. Second, the Boolean functions for the outputs of gates that receive only external inputs are derived:

$$T_1 = (AD)' = A' + D'$$

$$T_2 = (CD')' = C' + D$$

The second form follows directly from DeMorgan's theorem and may, at times, be more convenient to use. Third, Boolean functions of gates which have inputs from previously derived functions are determined in consecutive order until the output is expressed in terms of external input variables:

$$\begin{aligned} T_3 &= (C'T_1)' = (A'C' + C'D')' = (A + C)(C + D) = AC + AD + C + CD \\ &= C + AD \end{aligned}$$

$$T_4 = (BT_3)' = (B(C + AD))'$$

$$F = (T_4T_2)' = ((B(C + AD))'(CD'))' = B(C + AD) + CD'$$

Derivation of the Truth Table

The procedure for obtaining the truth table directly from a logic diagram is also outlined in Section 6.3. This procedure is demonstrated for the NAND logic diagram of Figure 6.13. First, the four input variables, together with their 16 combinations of 1s and 0s, are listed in Table 6.3. Second, the outputs of all gates are labeled with arbitrary symbols as in Figure 6.13. Third, we obtain the truth table for the outputs of those gates that are a function of the external input variables only. These are T_1 and T_2 . $T_1 = (AD)'$, so we mark 0s in those rows where both A and D are equal to 1 and fill the rest of the rows of T_1 with 1s. Also, $T_2 = (CD')'$, so we mark 0s in those rows where $C = 1$ and $D = 0$, and fill the rest of the rows of T_2 with 1s.

We then proceed to obtain the truth table for the outputs of those gates that are a function of previously defined outputs until the column for the output F is determined.

Table 6.3 Truth table for the circuit of Figure 6.13.

✓

$A B C D$	T_1	T_2	T_3	T_4	F
0 0 0 0	1	1	0	1	0
0 0 0 1	1	1	0	1	0
0 0 1 0	1	0	1	1	1
0 0 1 1	1	1	1	1	0
0 1 0 0	1	1	0	1	0
0 1 0 1	1	1	0	1	0
0 1 1 0	1	0	1	0	1
0 1 1 1	1	1	1	0	1
1 0 0 0	1	1	0	1	0
1 0 0 1	0	1	1	1	0
1 0 1 0	1	0	1	1	1
1 0 1 1	0	1	1	1	0
1 1 0 0	1	1	0	1	0
1 1 0 1	0	1	1	0	1
1 1 1 0	1	0	1	0	1
1 1 1 1	0	1	1	0	1

It is now possible to obtain an algebraic expression for the output from the derived truth table. The K-map shown in Figure 6.14 is obtained directly from Table 6.3. The simplified expression obtained from the K-map is

$$F = BC + ABD + CD' = B(C + AD) + CD'$$

This is the same as the expression of Figure 6.11, thus verifying the correct answer.

		CD			
		00	01	11	10
AB	00	0	0	0	1
	01	0	0	1	1
	11	0	1	1	1
	10	0	0	0	1

Figure 6.14 Derivation of F from Table 6.3 using K-map.

Conversion of NAND Logic Diagram to AND-OR Logic Diagram

It is sometimes convenient to convert a NAND logic diagram to its equivalent AND-OR logic diagram to facilitate the analysis procedure. By doing so, the Boolean function can be derived more easily without employing DeMorgan's theorem. The conversion of logic diagram is accomplished through a process reverse from that used for implementation. In Figure 4.8, we showed two alternate graphic symbols

for the NAND gate. By judicious use of both symbols, it is possible to convert a NAND diagram to an equivalent AND-OR form.

The conversion of a NAND logic diagram to an AND-OR diagram is achieved through a change in symbols from AND-invert to invert-OR in alternate levels of gates. The first level to be changed to an invert-OR symbol should be the last level. These changes produce pairs of circles along the same line, and these can be removed since they represent double complementation. Moreover, a one-input AND or OR gate can be removed since it does not perform a logical function. A one-input AND or OR with a circle in the input or output is changed to an inverter.

This procedure is demonstrated in Figure 6.15. The NAND logic diagram of Figure 6.15(a) is that of Figure 6.11(c) and is to be converted to an AND-OR logic diagram. The symbol of the gate in the last level is changed to an invert-OR. Looking for alternate levels, we find one gate requiring a change of symbol as shown in Figure 6.15(b). Any two circles along the same line are removed. Circles that go to external inputs are also removed and the corresponding input variable is complemented. The required AND-OR logic diagram is drawn in Figure 6.15(c).

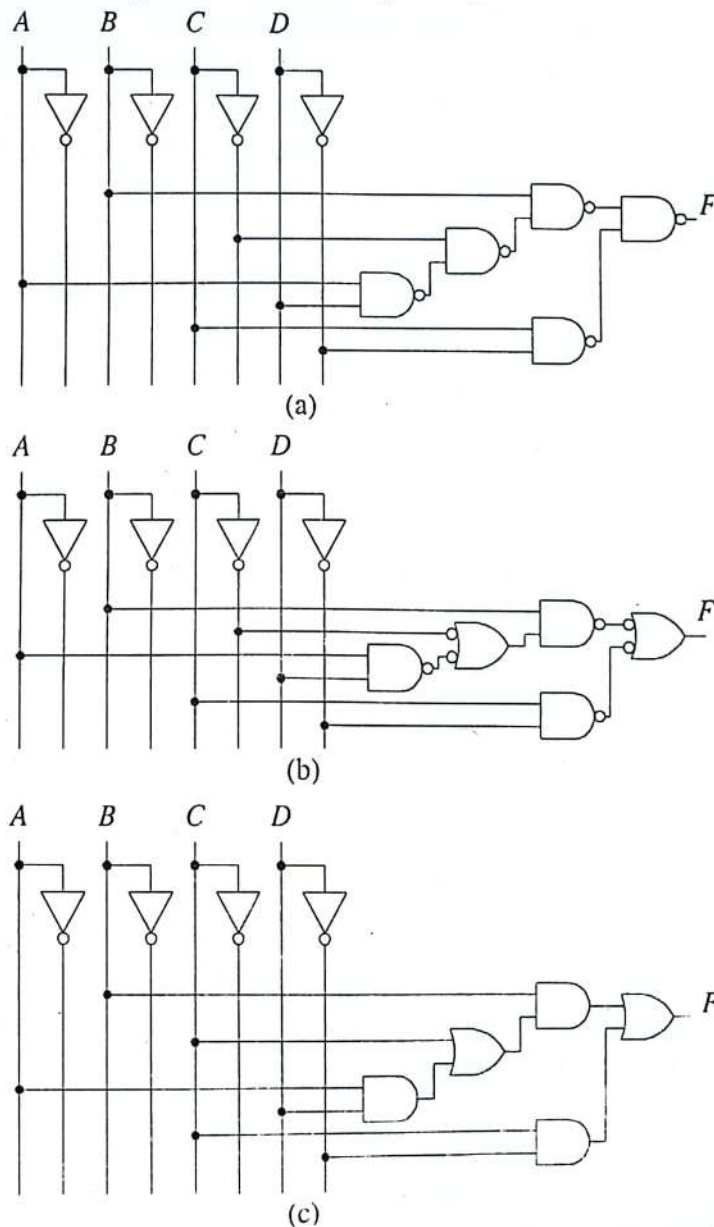


Figure 6.15 Conversion of NAND logic diagram to AND-OR logic diagram

6.7 Multi-Level NOR Circuits

The NOR function is the dual of the NAND function. For this reason, procedures and rules for multi-level NOR logic form a dual of the corresponding procedures and rules developed for multi-level NAND logic.

6.7.1 Boolean Function Implementation with Multi-Level NOR Circuit

The block diagram procedure for implementing Boolean function with multi-level NOR circuits is similar to the procedure outlined in the previous section for NAND gates.

1. From the given algebraic expression, draw the logic diagram with AND, OR, and NOT gates. Assume that both the uncomplemented and the complemented inputs are available.
2. Draw a second logic diagram with the equivalent NOR logic, as given in Figure 4.7, substituted for each AND, OR, and NOT gate.
3. Remove any two cascaded inverters from the diagram. Remove inverters connected to single external inputs and complement the corresponding input variable. The new logic diagram obtained is the required NOR gate implementation.

The procedure is illustrated for the function $F = B(C + AD) + CD'$.

1. The AND-OR implementation of this function is shown in the logic diagram of Figure 6.16(a).
2. For each AND gate, we substitute input inverters followed by a NOR gate; for each OR gate, we substitute a NOR gate followed by an inverter. This substitution follows directly from the logic equivalence of Figure 4.7 and is shown in the diagram of Figure 6.16(b).
3. The pair of cascaded inverters from the OR box to the AND box is removed. The five inverters connected to external inputs are removed and the input variables are complemented. The results is the NOR logic diagram shown in Figure 6.16(c).

The number of NOR gates in this example equals the number of AND-OR gates plus an additional inverter in the output (NOR gate 6). In general, the number of NOR gates required to implement a Boolean function equals the number of AND-OR gates, except for an occasional inverter. This is true provided both the uncomplemented and the complemented inputs are available.

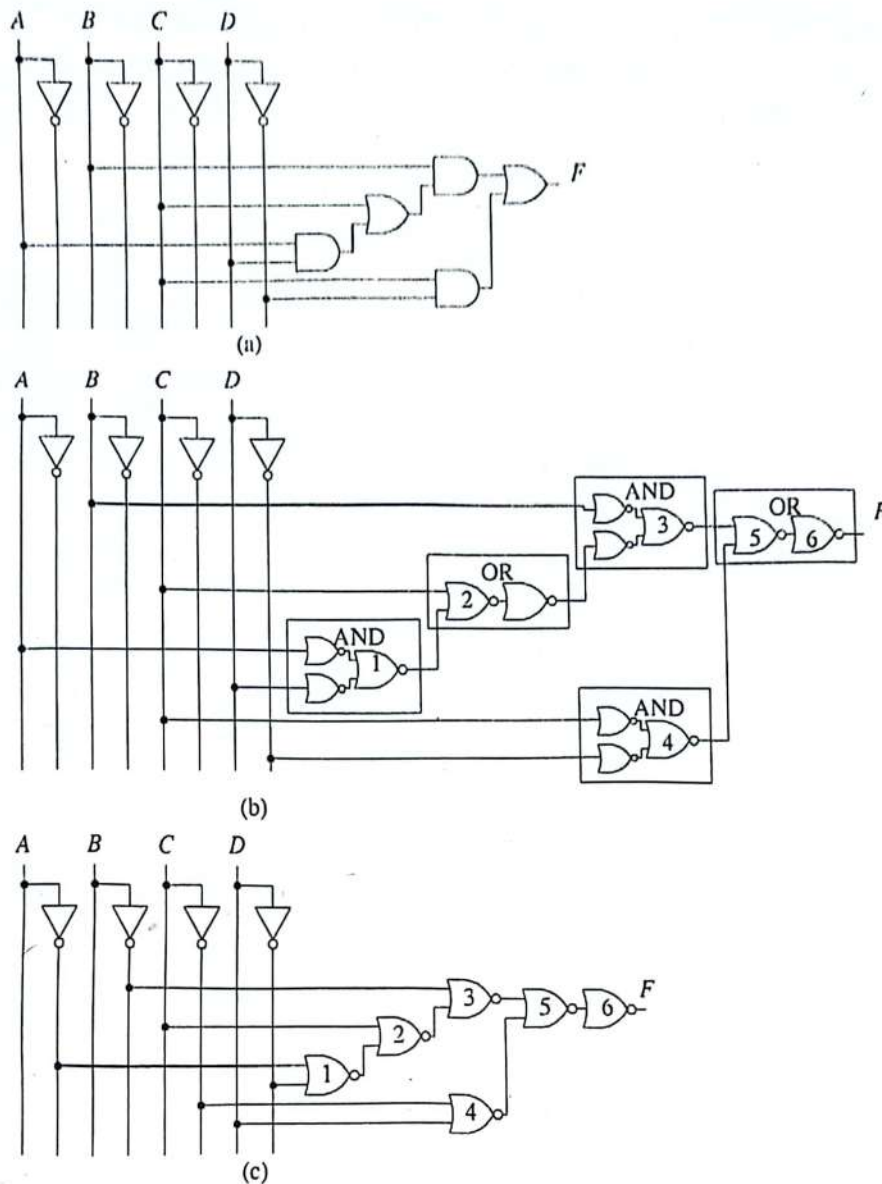


Figure 6.16 Implementation of $F = B(C + AD) + CD'$ with NOR gates.

6.7.2 Analysis of Multi-Level NOR Circuits

The analysis of NOR logic diagram follows the same procedure presented in Section 6.3.

Derivation of the Boolean Expression by Algebraic Manipulation

The procedure for deriving the Boolean function from a NOR logic diagram is demonstrated for the logic diagram of Figure 6.16(c), which is redrawn in Figure 6.17. First, all gate outputs are labeled with arbitrary symbols. Second, the Boolean functions for the outputs of gates that receive only external inputs are derived:

$$T_1 = (A' + D')' = AD$$

$$T_2 = (C' + D)' = CD'$$

Third, Boolean functions of gates which have inputs from previously derived functions are determined in consecutive order until the output is expressed in terms of input variables:

$$T_3 = (C + T_1)' = (C + AD)'$$

$$T_4 = (B' + T_3)' = (B' + (C + AD)')' = B(C + AD)$$

$$T_5 = (T_4 + T_2)' = (B(C + AD) + CD')'$$

$$F = T_5' = B(C + AD) + CD'$$

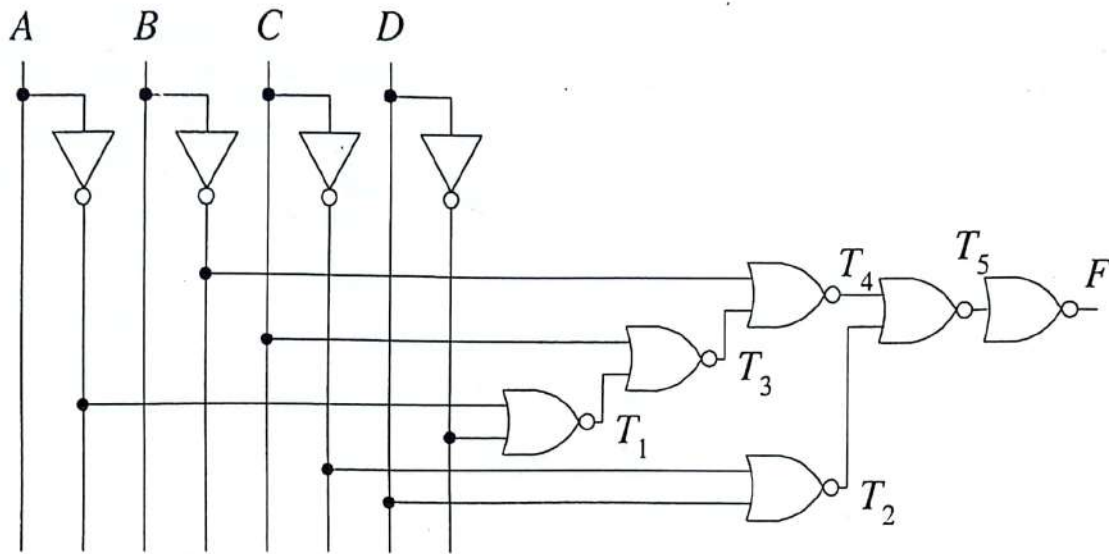


Figure 6.17 Analysis example of NOR circuit.

Derivation of the Truth Table

To obtain the truth table directly from the logic diagram of Figure 6.17, first the four input variables, together with their 16 combinations of 1s and 0s, are listed in Table 6.4. Second, the outputs of all gates are labeled with arbitrary symbols as in Figure 6.17. Third, we obtain the truth table for the outputs of those gates that are a function of the external input variables only. These are T_1 and T_2 . $T_1 = (A' + D')'$, so we mark 0s in those rows where $A = 0$ or $D = 0$ and fill the rest of the rows of T_1 with 1s. Also, $T_2 = (C' + D)'$, so we mark 0s in those rows where $C = 0$ or $D = 1$, and fill the rest of the rows of T_2 with 1s. We then proceed to obtain the truth table for the outputs of those gates that are a function of previously defined outputs until the column for the output F is determined.

Table 6.4 Truth table for the circuit of Figure 6.17.

$A B C D$	T_1	T_2	T_3	T_4	T_5	F
0 0 0 0	0	0	1	0	1	0
0 0 0 1	0	0	1	0	1	0
0 0 1 0	0	1	0	0	0	1
0 0 1 1	0	0	0	0	1	0
0 1 0 0	0	0	1	0	1	0
0 1 0 1	0	0	1	0	1	0
0 1 1 0	0	1	0	1	0	1
0 1 1 1	0	0	0	1	0	1
1 0 0 0	0	0	1	0	1	0
1 0 0 1	1	0	0	0	1	0
1 0 1 0	0	1	0	0	0	1
1 0 1 1	1	0	0	0	1	0
1 1 0 0	0	0	1	0	1	0
1 1 0 1	1	0	0	1	0	1
1 1 1 0	0	1	0	1	0	1
1 1 1 1	1	0	0	1	0	1

The truth table for F obtained in Table 6.4 is the same as the truth table of F in Table 6.3. The algebraic expression can be obtained as in Figure 6.14.

Conversion of NOR Logic Diagram to AND-OR Logic Diagram

To convert a NOR logic diagram to its equivalent AND-OR logic diagram, we use the two symbols for NOR gate shown in Figure 4.8. The OR-invert is the normal symbol for a NOR gate and the invert-AND is a convenient alternative that utilizes DeMorgan's theorem and the convention that small circles at the inputs denote complementation.

The conversion of a NOR logic diagram to an AND-OR diagram is achieved through a change in symbols from OR-invert to invert-AND starting from the last level and in alternate levels. Pairs of small circles along the same line are removed. A one-input AND or OR gate is removed, but if it has a small circle at the input or output, it is converted to an inverter.

This process is demonstrated in Figure 6.18, where the NOR logic diagram in Figure 6.18(a) is converted to an AND-OR diagram. The symbol of the gate in the last level (level 5) is changed to an invert-AND. Looking for alternate levels, we find two gates in level 3 and one in level 1. These three gates undergo a symbol change as shown in Figure 6.18(b). Any two circles along the same line are removed. Circles that go to external inputs are also removed and the corresponding input variable is complemented. The gate in level 5 becomes a one-input AND gate and is removed. The required AND-OR logic diagram is drawn in Figure 6.18(c).

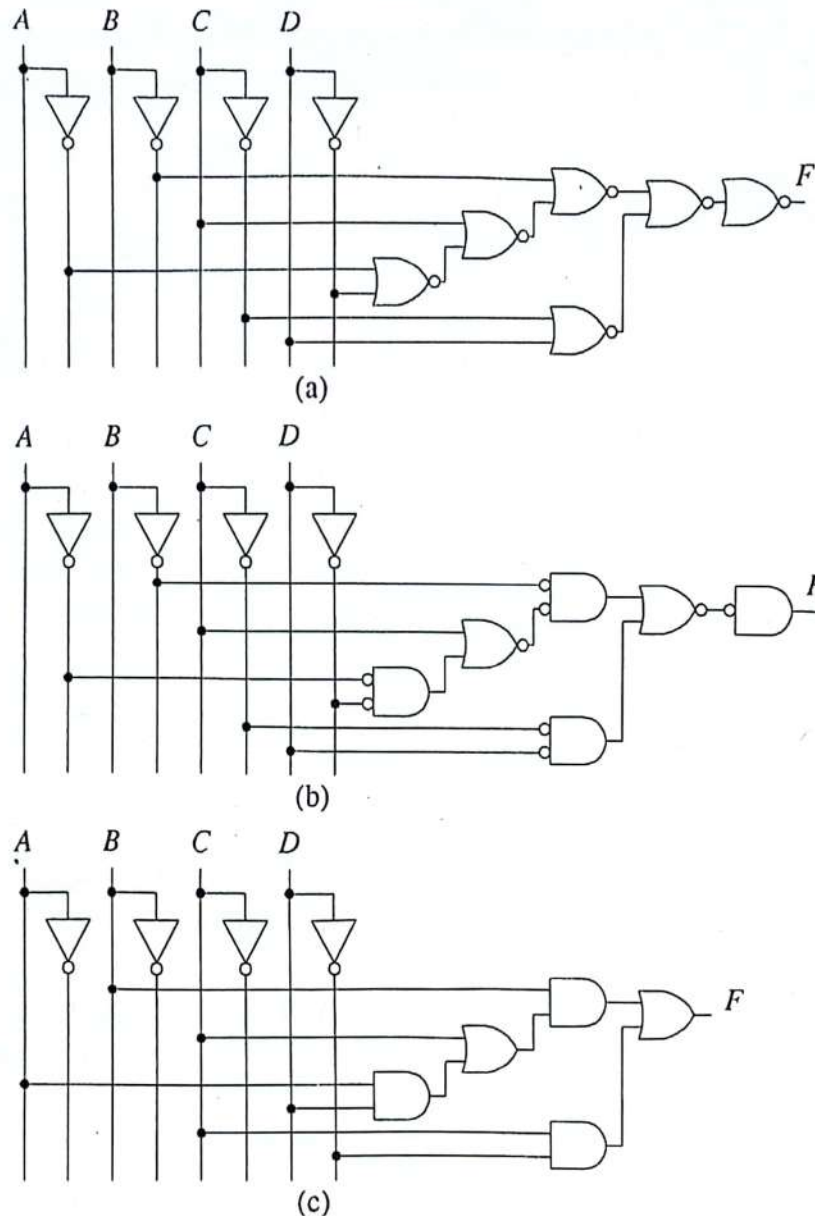


Figure 6.18 Conversion of NOR logic diagram to AND-OR logic diagram.

6.8 Two-Level AND-EXOR Logic Circuit

In conventional logic design a Boolean function is simplified in a sum of products form and the simplified function is implemented using two-level AND-OR circuit. It has been observed that logic implementation with EXOR gates exhibits many fold advantages over conventional OR-based logic implementation. One of such advantages to be noted is that, in many cases, the number of AND gates and the total number of input connections required for EXOR-based logic implementation is less than or at most equal to the number of AND gates and the total number of input connections required for OR-based logic implementation.

The OR and EXOR functions are introduced in Chapter 4. The truth tables for OR and EXOR functions are given in Figure 4.1 and are repeated in Table 6.5 for convenience.

Table 6.5 Truth table for OR and EXOR functions

$x y$	$F = x + y$	$F = x \oplus y$
0 0	0	0
0 1	1	1
1 0	1	1
1 1	1	0

From Table 6.5, we observe that OR and EXOR functions are the same for the second and the third rows of the truth table, that is, OR and EXOR functions are the same for exclusive inputs. Therefore, if A and B are two exclusive inputs or two exclusive product terms, we can write $A + B = A \oplus B$. Similarly, if A , B , and C are mutually exclusive product terms, we can write $A + B + C = A \oplus B \oplus C$.

A canonical sum of products (CSOP) function is a sum of minterms. From Table 3.6 it can be seen that all the minterms are mutually exclusive, that is, at a time only one of the minterms can be present in the function. Therefore, the OR symbols of a canonical sum of products function can be replaced by EXOR symbols. This type of function is called canonical exclusive-OR sum of products (CESOP) function. For example, the function $F(A, B, C, D) = \sum(0, 1, 3, 6, 7, 11)$ can be expressed as

$$\begin{aligned} F(A, B, C, D) &= A'B'C'D' + A'B'C'D + A'B'CD + A'BCD' + A'BCD + AB'CD \\ &= A'B'C'D' \oplus A'B'C'D \oplus A'B'CD \oplus A'BCD' \oplus A'BCD \oplus AB'CD \end{aligned}$$

As another example, consider the function

$$F(A, B, C, D) = A'CD' + A'C'D + AB'C'$$

The product terms $A'CD'$, $A'C'D$, and $AB'C'$ are mutually exclusive, that is, only one product term can be present at a time in the function. Therefore, the function can be written as

$$F(A, B, C, D) = A'CD' + A'C'D + AB'C' = A'CD' \oplus A'C'D \oplus AB'C'$$

A set of product terms are *mutually exclusive* if all possible pairs of product terms of the set are pair wise exclusive. Two product terms are *pair wise exclusive* if their product results in a 0 value. For example, $A'CD' \cdot A'C'D = 0$. Therefore, $A'CD'$ and $A'C'D$ are pair wise exclusive. On a K-map, the groups of 1s representing a set of mutually exclusive product terms are non-overlapping.

Simplification of Exclusive-OR Sum of Products Function

A canonical exclusive-OR sum of products function is an exclusive-OR sum of minterms. These minterms can be grouped on a K-map for simplification of exclusive-OR sum of products (ESOP) function. From the properties of OR function, we know that $A + A + \dots + A = A$. For this property, we can include a

1 in many overlapping groups of 1s during simplification of sum of products function using K-map. But the simplification of exclusive-OR sum of products function using K-map is different. From the properties of EXOR function, we know that $A \oplus A = 0$ but $A \oplus A \oplus A = A$. This property of EXOR function restricts us to include a 1 in only odd number of overlapping groups of 1s. This property also allows us to include a 0 within a group of 1s if that 0 is included even number of times. An even number of inclusion of a 0 will have no effect on the value of the function.

A two-level AND-EXOR circuit can be designed using the following procedure.

1. Represent the function on a K-map. Identify largest possible groups of cells such that a 1 is included odd number of times and a 0 is included even number of times.
2. Read the product terms for each group of cells. Combine the product terms by EXOR operators to form the simplified exclusive-OR sum of products function.
3. Draw the logic diagram for the minimized function.

The procedure is demonstrated using the following examples.

✓ **Example 1.** The function is $F(A, B, C, D) = \sum(0, 1, 3, 6, 7, 11)$.

1. The function is represented on the K-map of Figure 6.19(a). The groups of cells are identified as groups 1, 2, and 3. The cell 0010 containing 0 is included in groups 1 and 2, that is, two times. The cell 0011 containing 1 is included in all three groups, that is, three times.
2. The product terms for group 1 is $A'B'$; for group 2 is $A'C$; and for group 3 is $B'CD$. The simplified exclusive-OR sum of products function is

$$F(A, B, C, D) = A'B' \oplus A'C \oplus B'CD$$

3. The AND-EXOR logic diagram is drawn in Figure 6.19(b).

The corresponding sum of products simplification of the function is shown in Figure 6.19(c), which gives

$$F(A, B, C, D) = A'B'C' + B'CD + A'BC$$

The AND-OR logic diagram is drawn in Figure 6.19(d). Comparing logic diagrams of Figures 6.19(b) and (d), we see that both implementations require three AND gates, but the AND-EXOR implementation requires a total of 7 input connections, whereas the AND-OR implementation requires a total of 9 input connections.

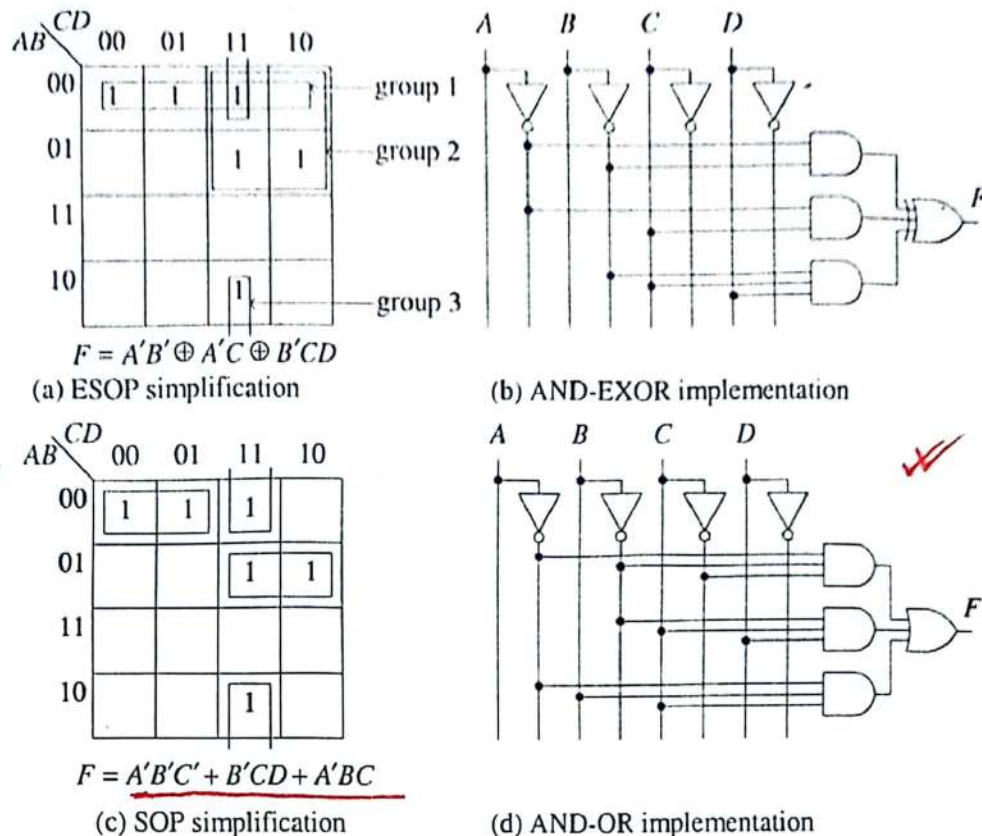


Figure 6.19 AND-EXOR and AND-OR implementations of the function

$$F(A, B, C, D) = \sum(0, 1, 3, 6, 7, 11).$$

Example 2. The function is $F(A, B, C, D) = \sum(0, 1, 3, 6, 7, 8, 10, 13)$.

1. The function is represented on the K-map of Figure 6.20(a). The groups of cells are identified as groups 1, 2, 3, and 4. The cell 0010 containing 0 is included in groups 1 and 2; the cell 0100 containing 0 is included in groups 1 and 3; the cell 0101 containing 0 is included in groups 1 and 4; and the cell 0000 containing 1 is included in groups 1, 2, and 3.
2. The product term for group 1 is A' ; for group 2 is $B'D'$; for group 3 is $A'C'D'$; and for group 4 is $BC'D$. The simplified exclusive-OR sum of products function is

$$F(A, B, C, D) = A' \oplus B'D' \oplus A'C'D' \oplus BC'D$$

3. The AND-EXOR logic diagram is drawn in Figure 6.20(b).

The corresponding sum of products simplification of the function is shown in Figure 6.20(c), which gives

$$F(A, B, C, D) = A'B'C' + A'B'D + A'BC + AB'D' + ABC'D$$

The AND-OR logic diagram is drawn in Figure 6.20(d). Comparing logic diagrams of Figures 6.20(b) and (d), we see that AND-EXOR implementation requires three

AND gates and a total of 9 input connections, whereas the AND-OR implementation requires five AND gates and a total of 16 input connections.

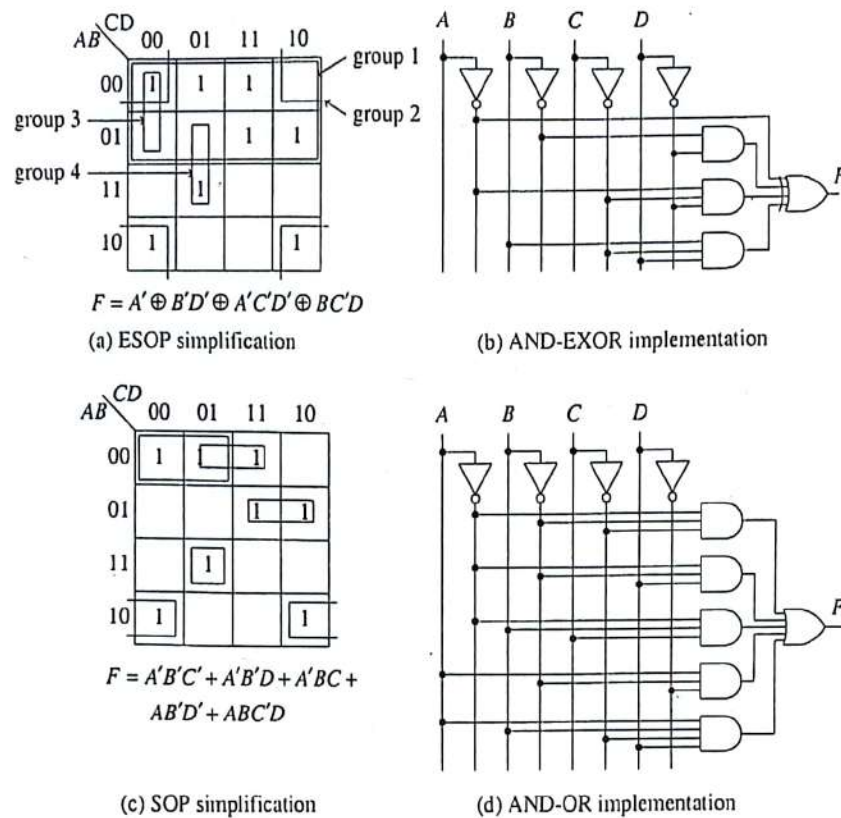


Figure 6.20 AND-EXOR and AND-OR implementations of the function $F(A, B, C, D) = \sum(0,1,3,6,7,8,10,13)$.

6.9 Three-Level AND-OR-EXOR Logic Circuits

In many cases, AND-EXOR circuits require fewer AND gates and fewer input connections than AND-OR circuits. However, from hardware point of view, multi-input EXOR gate is more expensive than multi-input OR gate. Therefore, it is desirable to limit the input of EXOR gates to as minimum as possible. This is achieved by introducing OR gates between AND gates and EXOR gate. The following procedure converts an AND-EXOR circuit into a three-level AND-OR-EXOR circuit without increasing the number of AND gates.

1. Partition the product terms of the exclusive-OR sum of products function into groups of mutually exclusive product terms. Replace the EXOR operators of each mutually exclusive group by OR operators.
2. If possible, simplify the newly formed sum of products expression.
3. Draw logic diagram using AND, OR, and EXOR gates.

The procedure is demonstrated using the following examples.

Example 1. The exclusive-OR sum of products function is $F(A, B, C, D) = A'C \oplus AD \oplus AB' \oplus BC'D \oplus B'CD'$.

1. The product terms of the exclusive-OR sum of products function are partitioned into groups of mutually exclusive product terms as follows

$$F(A, B, C, D) = (A'C \oplus AD) \oplus (AB' \oplus BC'D) \oplus B'CD'$$

After replacing the EXOR operators of the mutually exclusive groups of product terms, the function becomes

$$F(A, B, C, D) = (A'C + AD) \oplus (AB' + BC'D) \oplus B'CD'$$

2. The newly formed sum of products expression can not be simplified further as shown in the K-maps of Figure 6.21.

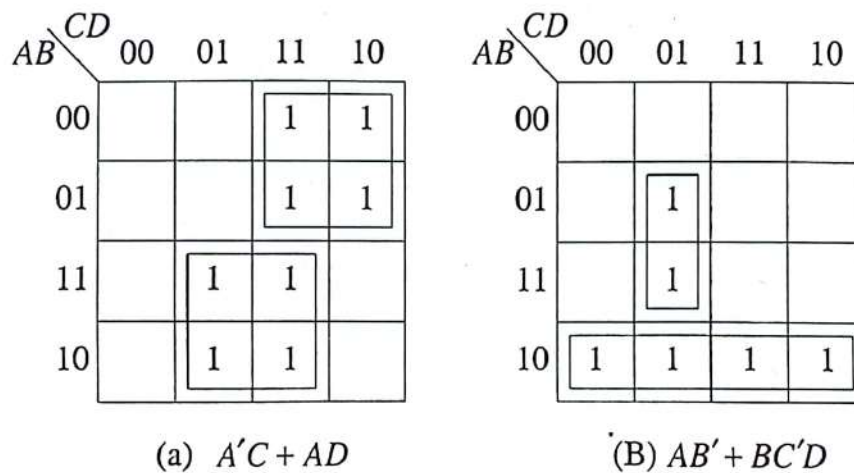


Figure 6.21 Simplification of (a) $A'C + AD$ and (b) $AB' + BC'D$.

3. The AND-OR-EXOR logic diagram is drawn in Figure 6.22.

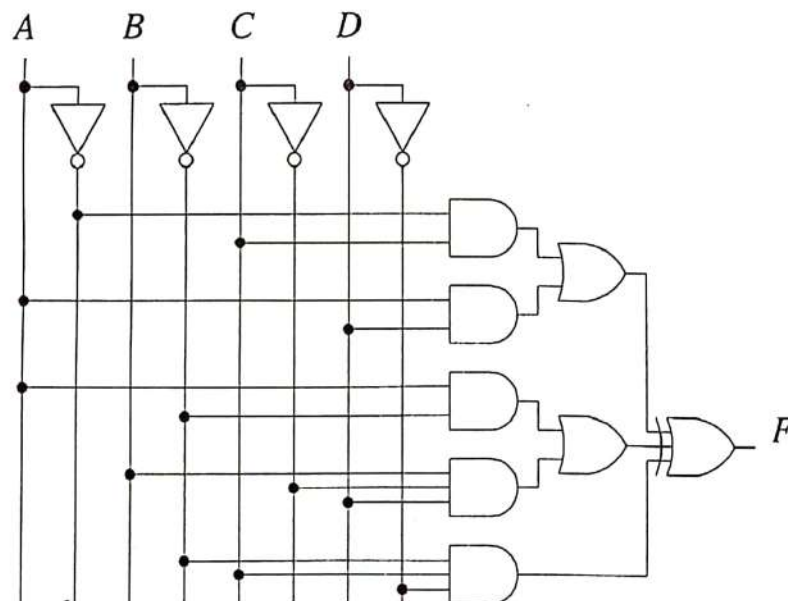


Figure 6.22 AND-OR-EXOR logic diagram for the function
 $F(A, B, C, D) = A'C \oplus AD \oplus AB' \oplus BC'D \oplus B'CD'$
 $= (A'C + AD) \oplus (AB' + BC'D) \oplus B'CD'$

Example 2. The exclusive-OR sum of products function is $F(A, B, C, D) = A'B' \oplus AB'D' \oplus ABCD' \oplus B'C$.

1. The product terms of the exclusive-OR sum of products function are partitioned into groups of mutually exclusive product terms as follows

$$F(A, B, C, D) = (A'B' \oplus AB'D' \oplus ABCD') \oplus B'C$$

After replacing the EXOR operators of the mutually exclusive group of product terms, the function becomes

$$F(A, B, C, D) = (A'B' + AB'D' + ABCD') \oplus B'C$$

2. The newly formed sum of products expression $A'B' + AB'D' + ABCD'$ can be simplified further as shown in the K-map of Figure 6.23, which gives

$$A'B' + AB'D' + ABCD' = A'B' + B'D' + ACD'$$

Therefore, the function becomes

$$F(A, B, C, D) = (A'B' + B'D' + ACD') \oplus B'C$$

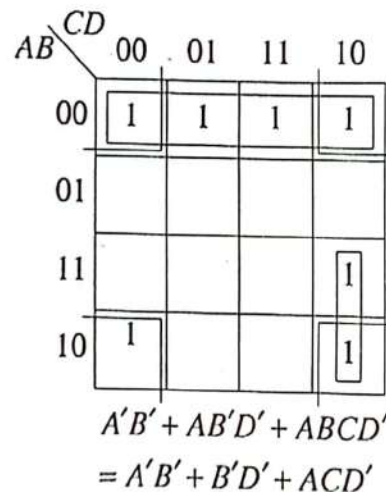


Figure 6.23 Simplification of $A'B' + AB'D' + ABCD'$.

3. The AND-OR-EXOR logic diagram is drawn in Figure 6.24.

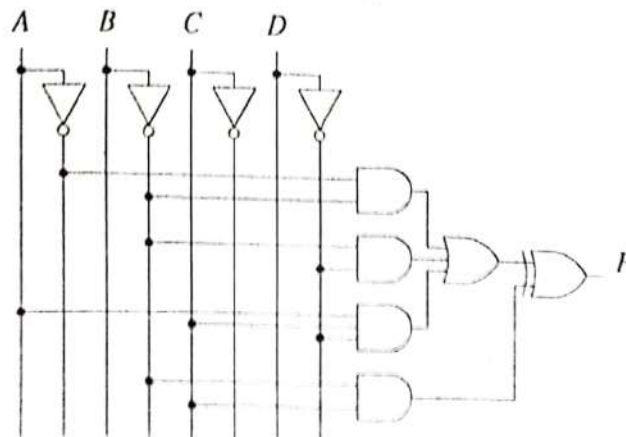


Figure 6.24 AND-OR-EXOR logic diagram for the function $F(A, B, C, D) = A'B' \oplus AB'D' \oplus ABCD' \oplus B'C = (A'B' + B'D' + ACD') \oplus B'C$

Exercises

- 6.1 Design a combinational logic circuit that has three inputs A , B , and C , and whose output will be high only when a majority of the inputs is high.
- 6.2 In a combinational logic circuit, four input lines A , B , C , and D are being used to represent a 4-bit binary number with A as the MSB and D as the LSB. The circuit produces a high output only when the binary number is greater than 0110_2 . Design this circuit.
- 6.3 Design a combinational logic circuit with inputs A , B , and C so that output F is high whenever $A = 0$ or whenever $B = C = 1$.
- 6.4 A combinational circuit has four inputs and one output. The output is equal to 1 when (1) all the inputs are equal to 1 or (2) none of the inputs are equal to 1 or (3) an odd number of inputs are equal to 1.
- 6.5 Design a combinational circuit that accepts a three-bit number and generates an output binary number equal to the square of the input number.
- 6.6 Design a combinational circuit with four input lines that represent a decimal digit in BCD and four output lines that generates the 9's complement of the input digit.
- 6.7 Design a combinational circuit whose input is a four-bit number and whose output is the 2's complement of the input number.
- 6.8 Design a combinational circuit that detects an error in the representation of a decimal digit in BCD. In other words, obtain a logic diagram whose output is 1 when the inputs contain an unused combination in the code.
- 6.9 Analyze the two-output combinational circuit shown in Figure 6.25.

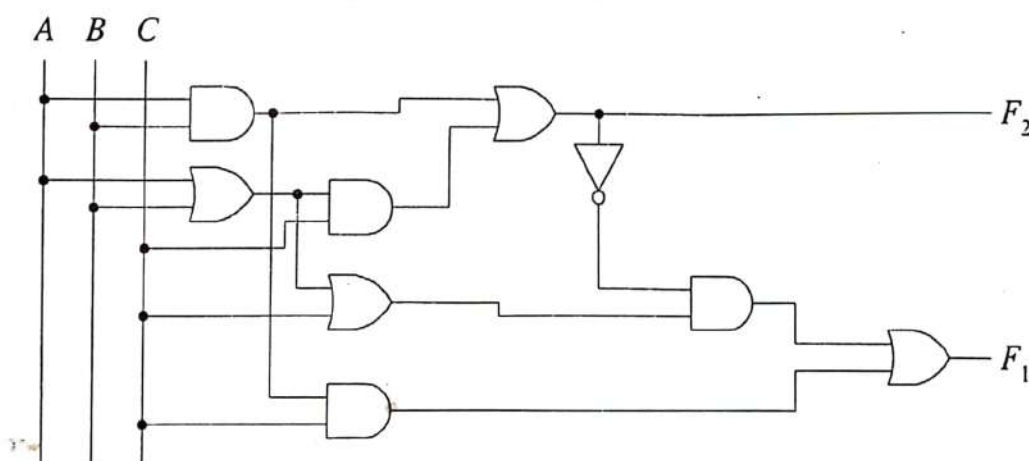


Figure 6.25 Logic diagram for analysis problem 6.9.

- 6.10** Simplify each of the following functions and implement them with NAND gates.
- (a) $F(A, B, C, D, E) = AC' + ACE + ACE' + A'CD' + A'D'E'$
- (b) $F(A, B, C, D) = (B' + D')(A' + C' + D)(A + B' + C' + D)(A' + B + C' + D')$

6.18 Determine the output Boolean functions of the circuits in Figure 6.28.

6.19 Obtain the truth tables for the circuits in Figure 6.28.

6.20 Obtain the equivalent AND-OR logic diagrams of Figure 6.28.

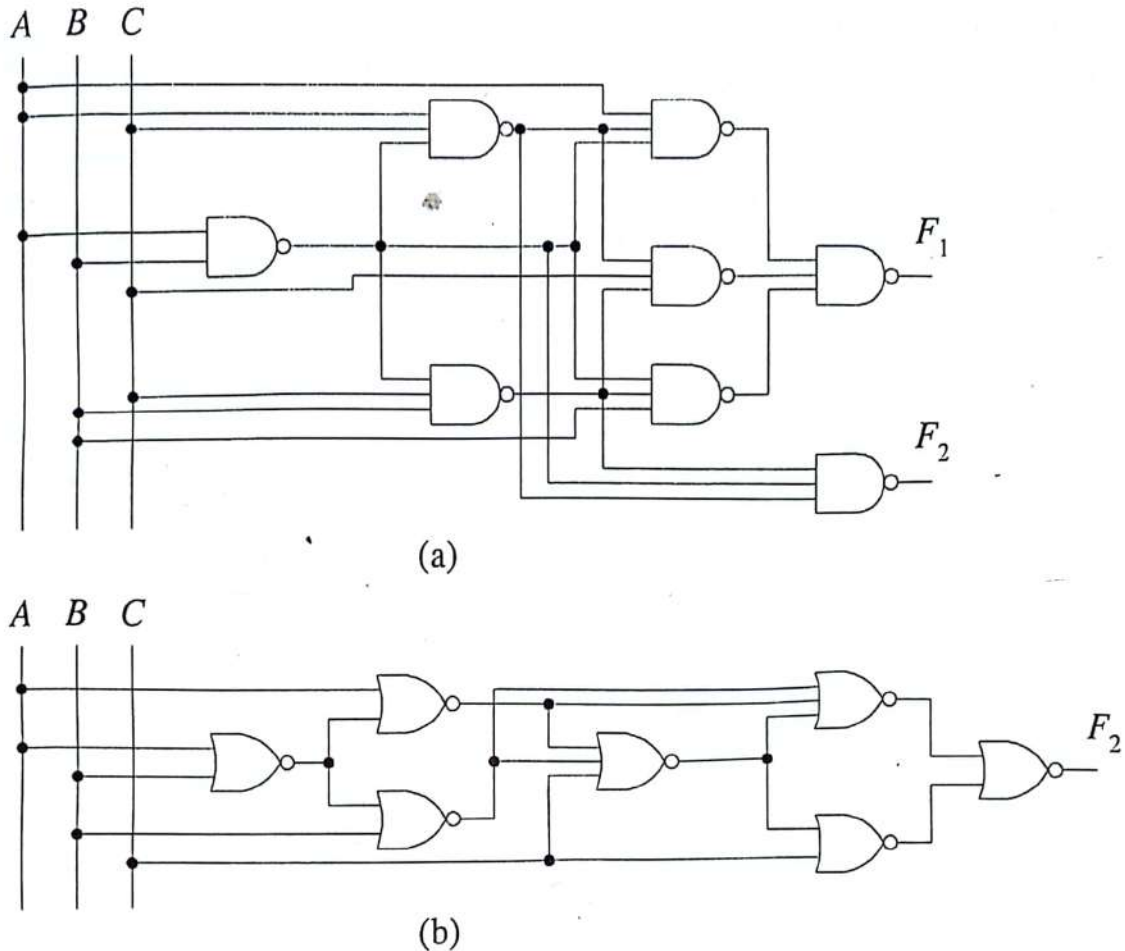


Figure 6.28 Logic diagrams for Problems 6.18, 6.19, and 6.20.

6.21 Design AND-EXOR circuits for the following functions

(a) $F(A, B, C, D) = \sum(0, 1, 8, 11)$

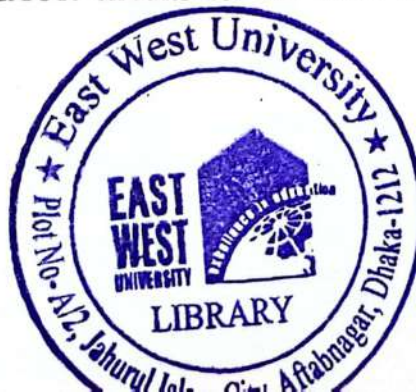
(b) $F(A, B, C, D) = \sum(1, 2, 4, 7, 9, 10, 13, 14)$

(c) $F(A, B, C, D) = \sum(0, 1, 3, 6, 7, 8, 10, 13)$

(d) $F(A, B, C, D) = \sum(0, 1, 2, 3, 8, 11)$

(e) $F(A, B, C, D) = \sum(0, 1, 3, 6, 7, 8, 11)$

6.22 Design AND-OR-EXOR circuits for the AND-EXOR circuits obtained in Problem 6.21.



Further Reading

1. E. J. McCluskey, *Logic Design Principles*, Prentice-Hall, Inc., 1986.
2. Zvi Kohavi, *Switching and Finite Automata Theory*, 2nd ed., McGraw-Hill Higher Education, 1990.
3. Tsutomu Sasao (editor), *Logic Synthesis and Optimization*, Kluwer Academic Publishers, 1993.
4. R. P. Jain, *Modern Digital Electronics*, 2nd ed., Tata McGraw-Hill Publishing Company Limited, India, 1998.
5. M. Morris Mano, *Digital Logic and Computer Design*, Prentice-Hall, Inc., 2000.
6. D. C. Green, *Digital Electronics*, 5th ed., Pearson Education, 2001.
7. Ronald J. Tocci and Neal S. Widmer, *Digital Systems: Principles and Applications*, 8th ed., Prentice-Hall, Inc., 2001.
8. V. K. Jain, *Switching Theory and Digital Electronics*, Khanna Publishers, India, 2002.
9. M. Morris Mano, *Digital Design*, 3rd ed., Pearson Education, 2002.



CHAPTER 7

ARITHMETIC AND COMPARATOR CIRCUITS

7.1 Introduction

Digital components perform a variety of information-processing tasks. Among the basic functions encountered are the various arithmetic operations and comparison. Digital circuits performing these operations are combinational circuits. These circuits may be designed using the design procedure for combinational logic presented in Chapter 6. This classical procedure assumes that, given two circuits that perform the same function, the one that requires fewer gates is preferable because it will cost less. This is not necessarily true when integrated circuits are used. Since several logic gates are included in a single IC package, it becomes economical to use as many of the gates from an already used package even if, by doing so, we increase the total number of gates. Moreover, some of the interconnections among gates in many ICs are internal to the chip and it is more economical to use as many internal interconnections as possible in order to minimize the number of wires between external pins. With integrated circuits, it is not the count of gates that determines the cost but the number and type of ICs employed and the number of external interconnections needed to implement the given function.

There are numerous occasions where the classical method of Chapter 6 will not produce the best combinational circuit for implementing a given function. Moreover, the truth table and the simplification procedure in this method become too cumbersome if the number of input variables is excessively large. The final circuit obtained dictates that it be implemented with a random connection of SSI gates, which may require a relatively large number of ICs and interconnecting wires. In many cases the application of an alternate design procedure can produce a combinational circuit for a given function which is far better than the one obtained by following the classical design method. The possibility of an alternate design procedure depends on the particular problem and the ingenuity of the designer. The classical method constitutes a general procedure that, if followed, guarantees to produce a result. However, before applying the classical method, it is always wise to investigate the possibility of an alternate method that may be more efficient for the particular problem at hand.

The first question that must be answered before going through a detailed design of a combinational circuit is whether the function is already available in an IC package. Numerous MSI devices are available commercially. These devices perform specific digital functions commonly employed in the design of digital systems. If an MSI device cannot be found to produce exactly the function needed, a resourceful designer may be able to formulate a method so as to incorporate an MSI device in this circuit. The selection of MSI components in preference to SSI gates is extremely important, since it would invariably result in a considerable reduction of IC packages and interconnecting wires.

In this chapter, the basic functions are designed using classical design method and some are designed using nonclassical design method. The more complicated functions are designed by incorporating MSI devices. After going through the design examples, a reader will be familiar with when to use classical method and when to use nonclassical method.

7.2 Adders

The most basic arithmetic operation is the addition of two binary digits. This simple addition consists of four possible elementary operations, namely, $0 + 0 = 0$, $0 + 1 = 1$, $1 + 0 = 1$, and $1 + 1 = 10$. The first three operations produce a sum whose length is one bit, but when both augend and addend bits are equal to 1, the binary sum consists of two bits. The higher significant bit of this result is called a *carry*. When the augend and addend numbers contain more significant bits, the carry obtained from the addition of two is added to the next higher-order pair of significant bits. A combinational circuit that performs the addition of two bits is called a *half-adder*. One that performs the addition of three bits (two significant bits and a previous carry) is a *full-adder*. The name of the former stems from the fact that two half-adders can be employed to implement a full-adder. The design of two adder circuits using classical method is presented next.

7.2.1 Half-Adder

From the verbal explanation of a half-adder, we find that this circuit needs two binary inputs and two binary outputs. The input variables designate the augend and addend bits; the output variables produce the sum and carry. We arbitrarily assign A and B to the two inputs and S (for sum) and C (for carry) to the outputs. The block diagram and the truth table of the half-adder are shown in Figure 7.1.

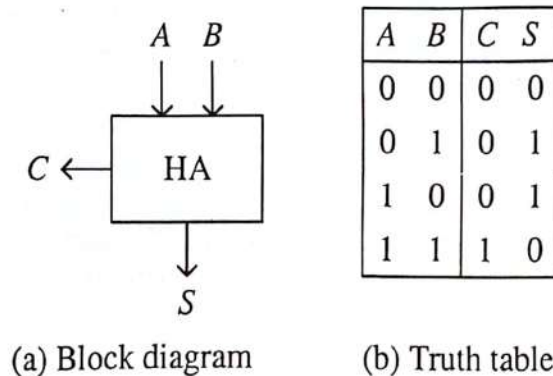


Figure 7.1 Block diagram and truth table of the half-adder.

The K-map simplification of the output functions of the half-adder is shown in Figure 7.2, which gives

$$S = A'B + AB' = A \oplus B$$

$$C = AB$$

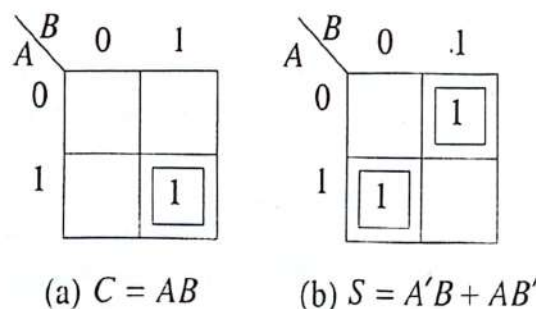


Figure 7.2 K-map simplification of output functions of the half-adder.

The implementation of the half-adder using AND-OR gates is shown in Figure 7.3(a) and using EXOR gate is shown in Figure 7.3(b).

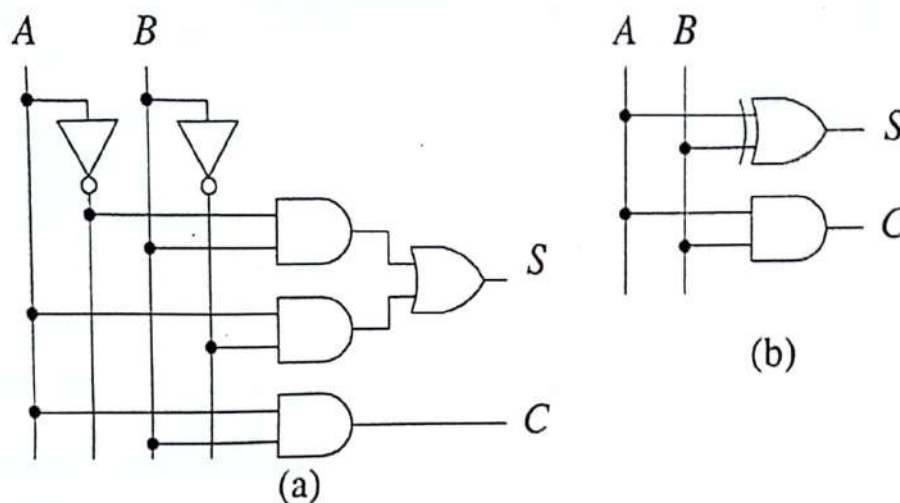


Figure 7.3 Implementation of half-adder.

7.2.2 Full-Adder

A full-adder is a combinational circuit that forms the arithmetic sum of three input bits. It consists of three inputs and two outputs. The block diagram of the full-adder is shown in Figure 7.4(a). Two of the input variables, denoted by A_i and B_i , represent the two significant bits to be added. The third input, C_i , represents the carry input from the previous lower significant position. The two outputs are designated by the symbols S_i for sum and C_{i+1} for carry output to the next higher significant position. The truth table of the full-adder is shown in Figure 7.4(b).

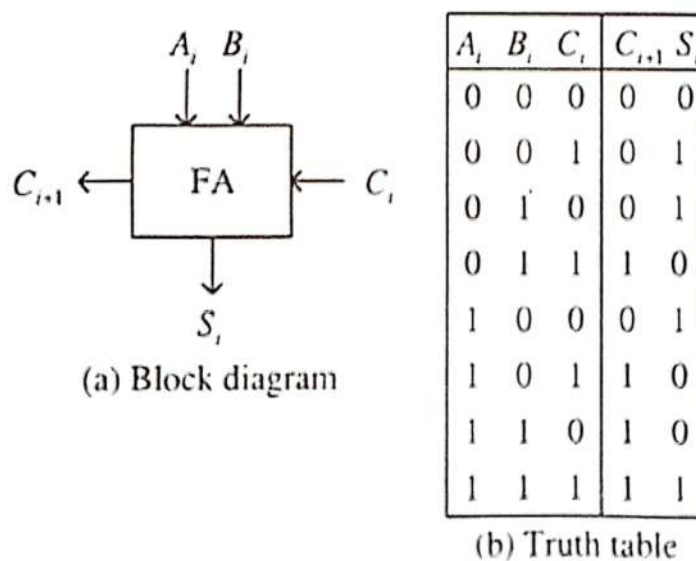
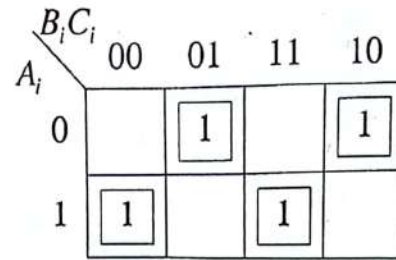
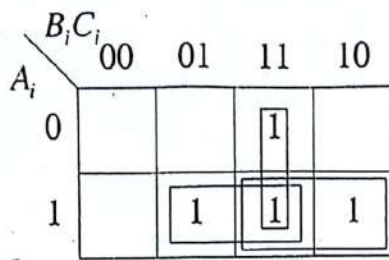


Figure 7.4 Block diagram and truth table for the full-adder.

The K-map simplification of the output functions of the full-adder is shown in Figure 7.5, which gives

$$S_i = A_i' B_i' C_i + A_i' B_i C_i' + A_i B_i' C_i' + A_i B_i C_i$$

$$C_{i+1} = A_i B_i + A_i C_i + B_i C_i$$



(a) $C_{i+1} = A_i B_i + A_i C_i + B_i C_i$ (b) $S_i = A_i' B_i' C_i + A_i' B_i C_i' + A_i B_i' C_i' + A_i B_i C_i$

Figure 7.5 K-map simplification of output functions of the full-adder.

The logic diagram for the full-adder implementation is shown in Figure 7.6.

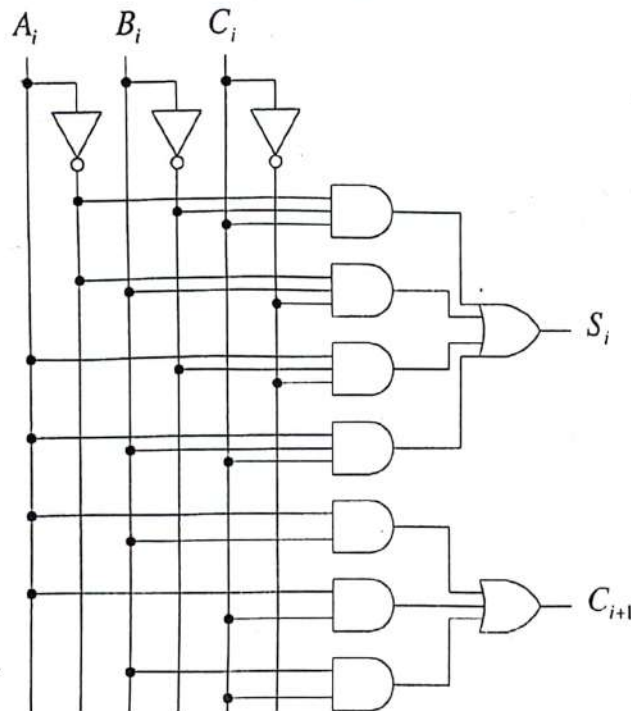
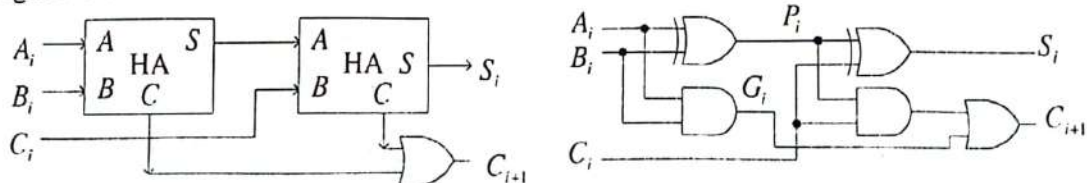


Figure 7.6 Implementation of the full-adder.

7.2.3 Full-Adder using Half-Adder

A full-adder can be implemented with two half-adders and one OR gate, as shown in Figure 7.7.



(a) Connection using block diagram

(b) Logic diagram

Figure 7.7 Implementation of a full-adder with two half-adders and an OR gate.

The output functions can be verified as follows:

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

The expression for S_i is

$$\begin{aligned} S_i &= P_i \oplus C_i \\ &= P_i C_i' + P_i' C_i \\ &= (A_i \oplus B_i) C_i' + (A_i \oplus B_i)' C_i \\ &= (A_i B_i' + A_i' B_i) C_i' + (A_i B_i + A_i' B_i') C_i \\ &= A_i B_i' C_i' + A_i' B_i C_i' + A_i B_i C_i + A_i' B_i' C_i \end{aligned}$$

The expression for C_{i+1} is

$$\begin{aligned} C_{i+1} &= G_i + P_i C_i \\ &= A_i B_i + (A_i \oplus B_i) C_i \\ &= A_i B_i + (A_i B_i' + A_i' B_i) C_i \\ &= A_i B_i + A_i B_i' C_i + A_i' B_i C_i \\ &= A_i B_i (C_i + C_i') + A_i B_i' C_i + A_i' B_i C_i \\ &= (A_i B_i C_i + A_i B_i C_i') + (A_i B_i' C_i + A_i' B_i C_i) + (A_i B_i C_i + A_i' B_i C_i) \\ &= A_i B_i + A_i C_i + B_i C_i \end{aligned}$$

These expressions verify the correctness of the full-adder implementation of Figure 7.7.

7.3 Subtractor

The subtraction of two binary numbers may be accomplished by taking the complement of the subtrahend and adding it to the minuend (Section 2.5). By this method, the subtraction operation becomes an addition operation requiring full-adder for its machine implementation. It is, however, possible to implement subtraction with logic circuits in a direct manner, as done with paper and pencil. By this method, each subtrahend bit of the number is subtracted from its corresponding significant minuend bit to form a difference bit. If the minuend bit is smaller than the subtrahend bit, a 1 is borrowed from the next significant position. The fact that a 1 has been borrowed must be conveyed to the next higher pair of bits by means of a binary signal coming out (output) of a given stage and going into (input) the next higher stage. Just as there are half- and full-adders, there are half- and full-subtractors.

7.3.1 Half-Subtractor

A half-subtractor is a combinational circuit that subtracts two bits and produces their difference. It also has an output to specify if a 1 has been borrowed. The block diagram of the half-subtractor is shown in Figure 7.8(a). We arbitrarily designate the minuend bit by X and the subtrahend bit by Y . To perform $X - Y$, we have to check the relative magnitudes of X and Y . If $X \geq Y$, we have three possibilities: $0 - 0 = 0$, $1 - 0 = 1$, and $1 - 1 = 0$. The results is called the *difference bit*. If $X < Y$, we have $0 - 1$, and it is necessary to borrow a 1 from the next higher stage. The 1 borrowed from the next higher stage adds 2 to the minuend bit. With the minuend equal to 2, the difference becomes $2 - 1 = 1$. The half-subtractor needs two outputs. One output generates the difference and will be designated by the symbol D . The second output, designated B for borrow, generates the binary signal that informs the next stage that a 1 has been borrowed. The truth table of the half-subtractor is shown in Figure 7.8(b).

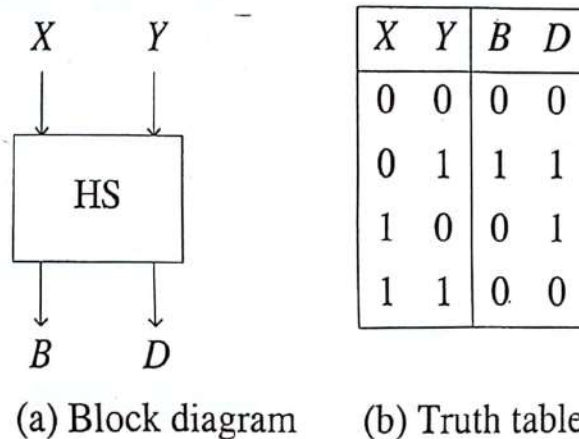


Figure 7.8 Block diagram and truth table of the half-subtractor.

The K-map simplification of the output functions of the half-subtractor is shown in Figure 7.9, which gives

$$D = X'Y + XY' = X \oplus Y$$

$$B = X'Y$$

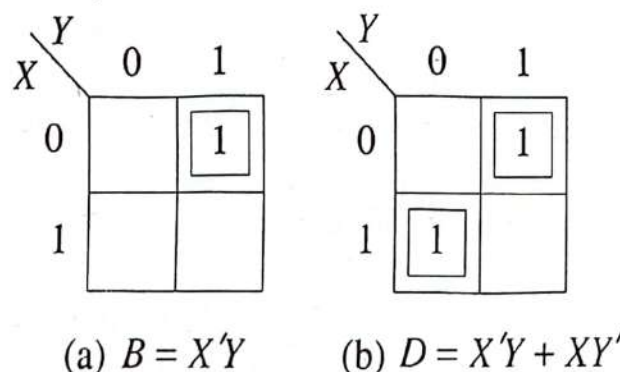


Figure 7.9 K-map simplification of the output functions of the half-subtractor. The implementation of the half-subtractor is shown in Figure 7.10.

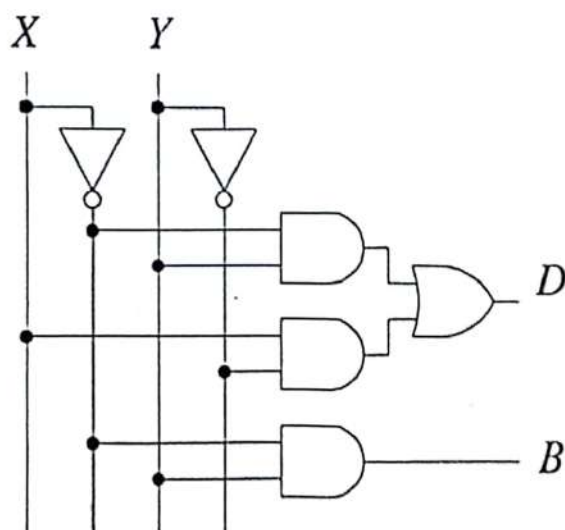


Figure 7.10 Implementation of the half-subtractor.

It is interesting to note that the logic for D is exactly the same as the logic for S output in the half-adder.

7.3.2 Full-Subtractor

A full-subtractor is a combinational circuit that performs a subtraction between two bits, taking into account that a 1 may have been borrowed by a lower significant stage. This circuit has three inputs and two outputs. The three inputs, X_i , Y_i , and B_i denote the minuend, subtrahend, and previous borrow, respectively. The two outputs, D_i and B_{i+1} , represent the difference and output borrow, respectively. The block diagram of the full-subtractor is shown in Figure 7.11(a) and its truth table is shown in Figure 7.11(b). The values of the output variables are determined from the subtraction of $X_i - Y_i - B_i$. The input combinations having input borrow $B_i = 0$ reduce to the same four conditions of the half-subtractor. For $X_i = 0$, $Y_i = 0$, and $B_i = 1$, we have to borrow a 1 from the next stage, which makes $B_{i+1} = 1$ and adds 2 to X_i . Since $2 - 0 - 1 = 1$, $D_i = 1$. For $X_i = 0$, $Y_i = 1$, and $B_i = 1$, we need to borrow again, making $B_{i+1} = 1$ and $X_i = 2$. Since $2 - 1 - 1 = 0$, $D_i = 0$. For $X_i = 1$, $Y_i = 0$, and $B_i = 1$, we have $X_i - Y_i - B_i = 0$, which makes $B_{i+1} = 0$ and $D_i = 0$. Finally, for $X_i = 1$, $Y_i = 1$, and $B_i = 1$, we have to borrow 1, making $B_{i+1} = 1$ and $X_i = 3$, and $3 - 1 - 1 = 1$, making $D_i = 1$.

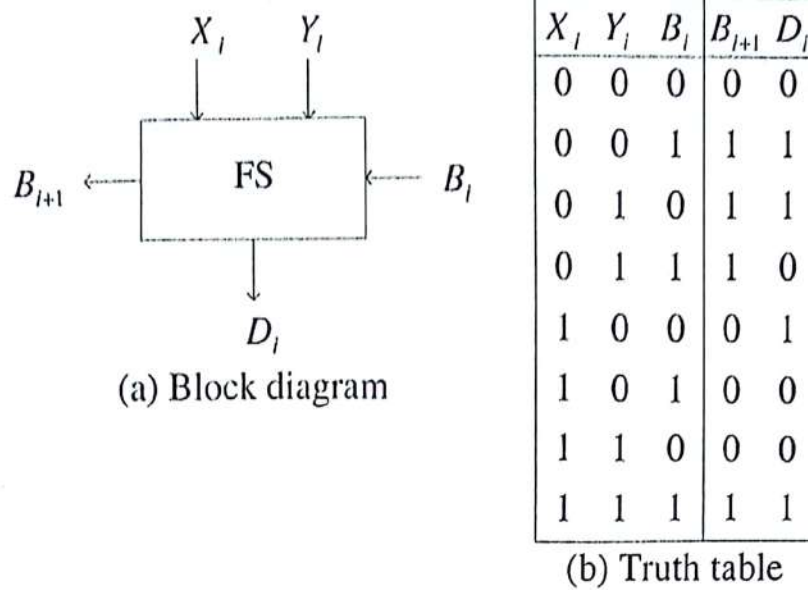


Figure 7.11 Block diagram and truth table of the full-subtractor.

The K-map simplification of the output functions is shown in Figure 7.12, which gives

$$D_i = X_i'Y_i'B_i + X_i'Y_iB_i' + X_iY_i'B_i' + X_iY_iB_i$$

$$B_{i+1} = X_i'Y_i + X_i'B_i + Y_iB_i$$

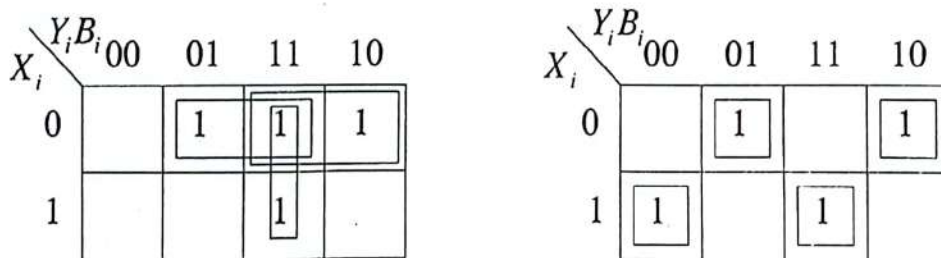


Figure 9.12 K-map simplification of the full-subtractor.

Again we note that the logic function for output D_i in the full-subtractor is exactly the same as output S_i in the full-adder. Moreover, the output B_{i+1} resembles the function for C_{i+1} in the full-adder, except that the input variable X_i is complemented. Because of these similarities, it is possible to convert a full-adder into a full-subtractor by merely complementing X_i prior to its application to the gates that form the carry output. The implementation of the full-subtractor is shown in Figure 7.13.

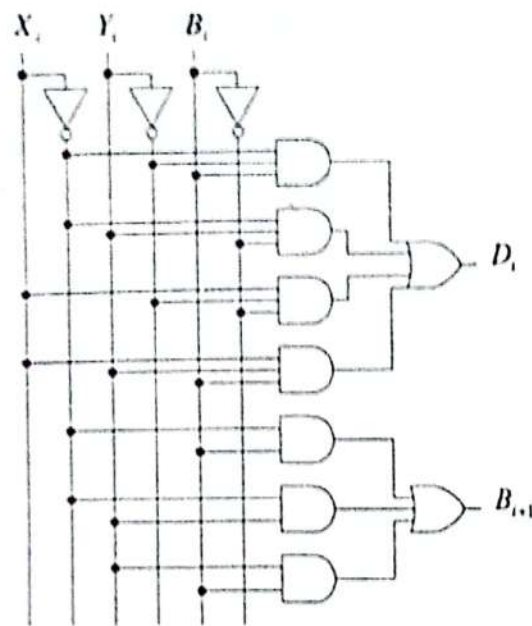


Figure 7.13 Implementation of the full-subtractor.

7.4 Binary Parallel Adder

The full-adder introduced in Section 7.2 forms the sum of two bits and a previous carry. Two binary numbers of n bits each can be added by means of this circuit. To demonstrate with a specific example, consider two binary numbers, $A = 1010$ and $B = 0011$, whose sum is $S = 1110$. When a pair of bits are added through a full-adder, the circuit produces a carry to be used with the pair of bits one significant position higher. This is shown below.

Subscript	3 2 1 0
Augend, A_i	1 0 1 1
Addend, B_i	0 0 1 1
Input Carry, C_i	0 1 1 0
Sum, S_i	1 1 1 0
Output Carry, C_{i+1}	0 0 1 1

The bits are added with full-adders, starting from the least significant position (subscript 0), to form the sum bit and carry bit. The input carry C_0 in the least significant position must be 0. The value of C_{i+1} is the output carry of the full-adder in the i th significant position. This value is transferred into the input carry of the full-adder that adds the bits in the $(i + 1)$ st significant position, that is, one higher significant position to the left. The sum bits are thus generated starting from the rightmost position and are available as soon as the corresponding previous carry bit is generated.

A *binary parallel adder* (or *binary adder* in short) is a digital function that produces the arithmetic sum of two binary numbers in parallel. It consists of full-adders connected in cascade, with the output carry from one full-adder connected to the input carry of the next full-adder. The block diagram of a 4-bit binary adder is shown in Figure 7.14(a) and its implementation using four full-adders is shown in Figure 7.14(b).

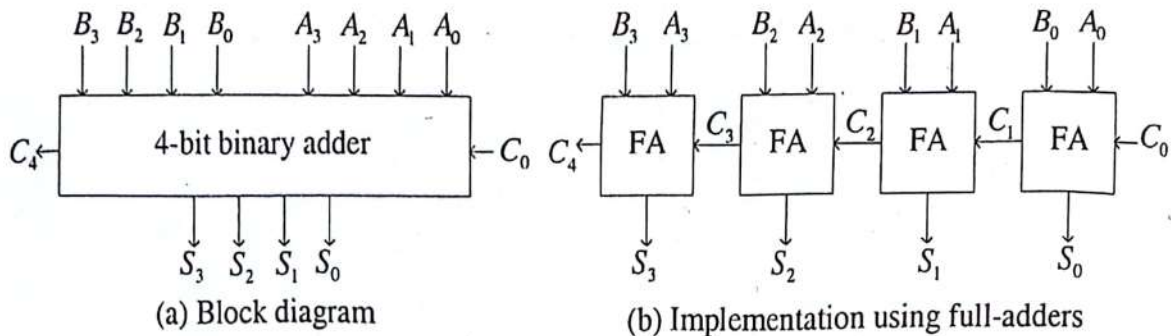


Figure 7.14 4-bit binary adder.

All bits of A and B are applied simultaneously. The output carry from one full-adder is connected to the input carry of the full-adder one position to its left. As soon as the carries are generated, the correct sum bits emerge from the sum outputs of all full-adders.

An n -bit binary adder requires n full-adders. It can be constructed from 4-bit, 2-bit, and 1-bit binary adder ICs by cascading several packages. The output carry from one package must be connected to the input carry of the one with the next higher-order bits. Cascading of two 4-bit binary adders to implement a 8-bit binary adder is shown in Figure 7.15. The output carry of the leftmost stage indicates the overflow, that is, a 1 in C_8 indicates that the sum exceeds 11111111.

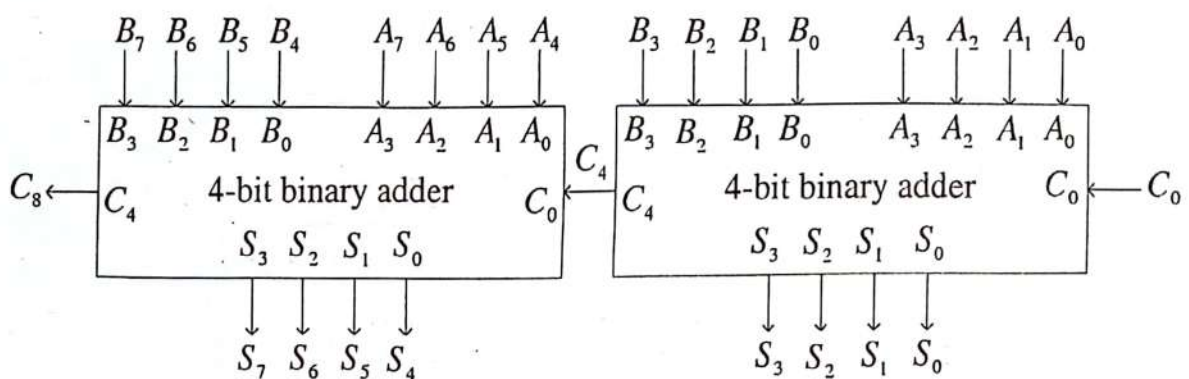


Figure 7.15 Cascading of two 4-bit binary adders to implement a 8-bit binary adder.

The 4-bit binary adder is a typical example of an MSI function. It can be used in many applications involving arithmetic operations. Observe that the design of this circuit by the classical method would require a truth table with $2^9 = 512$ entries, since there are nine inputs to the circuit. By using an iterative method of cascading an already known function, we were able to obtain a simple and well-organized implementation.

7.5 Binary Adders with Look-Ahead Carry

The addition of two binary numbers in parallel implies that all the bits of the augend and the addend are available for computation at the same time. As in any combinational circuit, the signal must propagate through the gates before the correct output sum is available in the output terminals. The total propagation time is equal to the propagation delay of a typical gate times the number of gate levels in the circuit. The longest propagation delay time in a parallel adder is the time it takes the carry to propagate through the full-adders. Since each bit of the sum output depends on the value of the input carry, the value of S_i in any given stage in the adder will be in its steady-state final value only after the input carry to that stage has been propagated. Consider output S_3 in Figure 7.14(b). Inputs A_3 and B_3 reach a steady value as soon as input signals are applied to the adder. But input carry C_3 does not settle to its final steady-state value until C_2 is available in its steady-state value. Similarly, C_2 has to wait for C_1 , and so on down to C_0 . Thus only after the carry propagates through all stages, the last output S_3 and carry C_4 will settle to their final steady-state value.

The number of gate levels for the carry propagation can be found from the circuit of the full-adder of Figure 7.7(b). The signals at P_i and G_i settle to their steady-state value after the propagation through their respective gates. These two signals are common to all full-adders and depend only on the input augend and addend bits. The signal from the input carry, C_i , to the output carry, C_{i+1} , propagates through an AND gate and an OR gate, which constitutes two gate levels. If there are four full-adders in the parallel adder, the output carry C_4 would have $2 \times 4 = 8$ gate levels from C_0 to C_4 . The total propagation time in the adder would be the propagation time in one half-adder plus eight gate levels. For an n -bit parallel adder, there are $2n$ gate levels for the carry to propagate through.

The carry propagation time is a limiting factor on the speed with which two numbers are added in parallel. Although a parallel adder, or any combinational circuit, will always have some value at its output terminals, the outputs will not be correct unless the signals are given enough time to propagate through the gates connected from the inputs to the outputs. Since all other arithmetic operations are implemented by successive additions, the time consumed during the addition process is very critical. An obvious solution for reducing the carry propagation delay times is to employ faster gates with reduced delays. But physical circuits have a limit to their capacity. Another solution is to increase the equipment complexity in such a way that the carry delay time is reduced. There are several techniques for reducing the carry propagation time in a parallel adder. The most widely used technique employs the principle of *look-ahead* carry.

Consider the circuit of the full-adder shown in Figure 7.7(b). The output sum and carry can be expressed as

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$

where

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

G_i is called a *carry generate* and it produces an output carry when both A_i and B_i are one, regardless of the input carry. P_i is called a *carry propagate* because it is the term associated with the propagation of the carry from C_i to C_{i+1} .

We now write the Boolean function for the carry output of each stage and substitute for each C_i its value from the previous equations.

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1$$

$$= G_1 + P_1 (G_0 + P_0 C_0)$$

$$= G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 C_2$$

$$= G_2 + P_2 (G_1 + P_1 G_0 + P_1 P_0 C_0)$$

$$= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 C_3$$

$$= G_3 + P_3 (G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0)$$

$$= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0.$$

Since the Boolean functions for each output carry is expressed in sum of products form, each function can be implemented with one level of AND gates followed by an OR gate. The circuit implementing these functions is called *look-ahead carry generator*. The block diagram and logic implementation of 4-bit look-ahead carry generator is shown in Figure 7.16. Note that C_4 does not have to wait for C_3 , C_2 , and C_1 to propagate; in fact, C_4 is propagated at the same time as C_3 , C_2 , and C_1 .

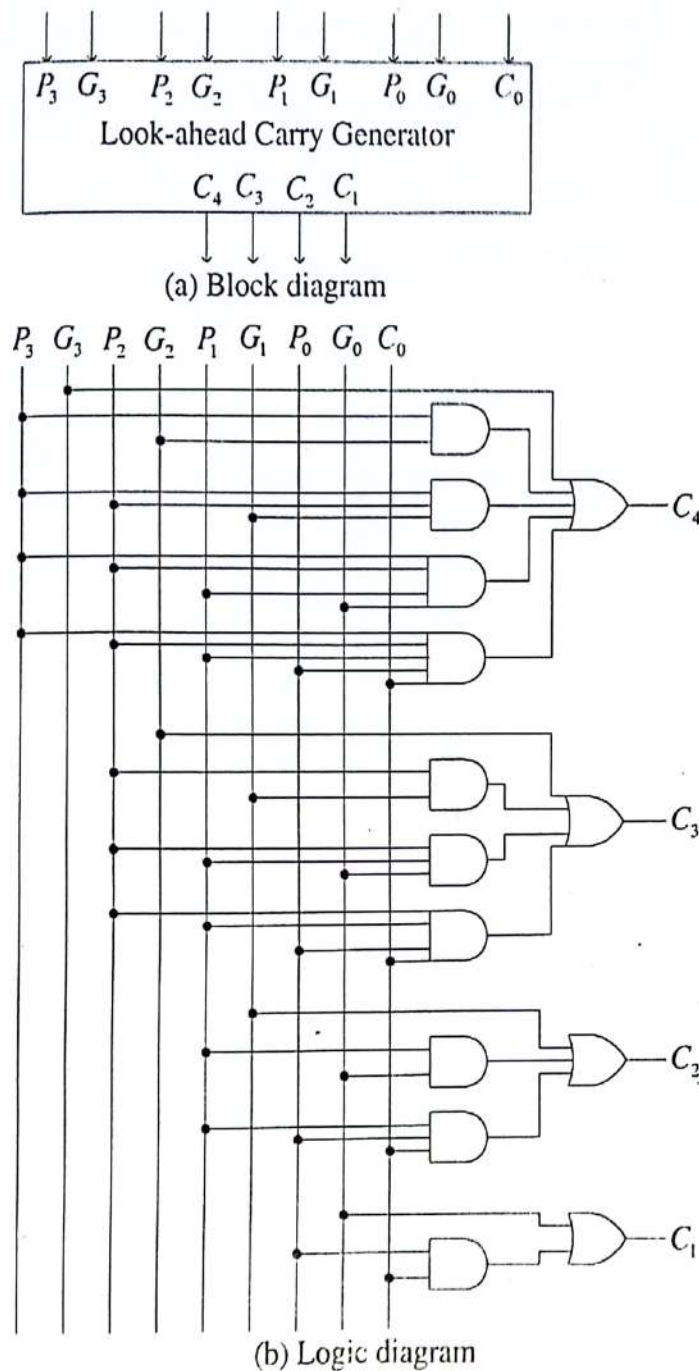


Figure 7.16 4-bit look-ahead carry generator.

The construction of a 4-bit binary adder with a look-ahead carry generator is shown in Figure 7.17. Each sum output requires two EXOR gates. The output of the first EXOR gate generates the P_i variable, and the AND gate generates the G_i variable. All the P s and G s are generated in one gate level. The carries are propagated through the look-ahead carry generator and applied as inputs to the second EXOR gate. After the P and G signals settle into their steady-state values, all output carries are generated after a delay of two levels of gates. Thus, outputs S_1 through S_3 have equal propagation delay times.

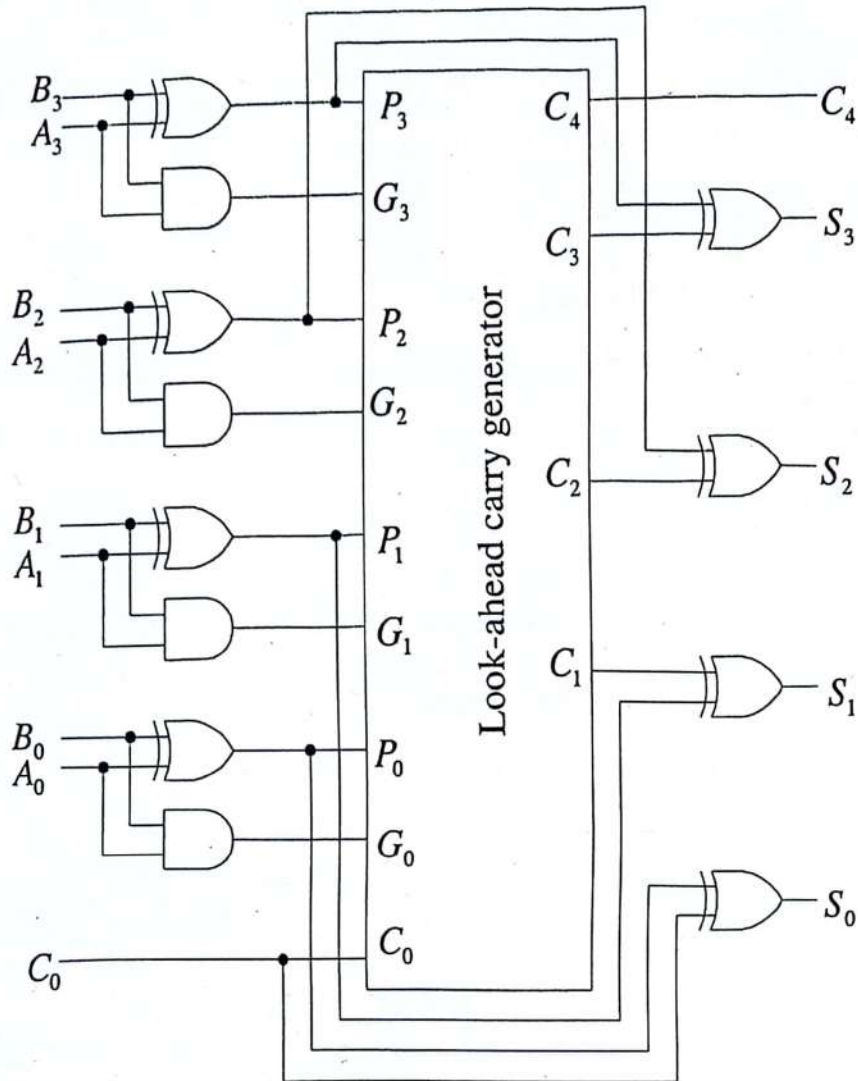


Figure 7.17 4-bit binary adder with look-ahead carry generator.

7.6 2's Complement Arithmetic Circuits

Most modern computers use 2's complement system to represent negative numbers and to perform subtraction. The operations of addition and subtraction of signed numbers can be performed using only the addition if we use the 2's complement form to represent negative numbers.

7.6.1 2's Complement Adder

Positive and negative numbers, including the sign bits, can be added together in the basic binary adder circuit of Figure 7.14 when the negative numbers are in 2's complement form. This is illustrated in Figure 7.18 for the addition of -3 and $+6$. The -3 is represented in its 2's complement form as 1101, where the first 1 is the sign bit; the $+6$ is represented in its 2's complement form as 0110, with the first 0 as the sign bit. The 4-bit binary adder produces sum outputs of 0011, which represent $+3$ in 2's complement form. The C_4 output is 1 but is disregarded in the 2's complement method.

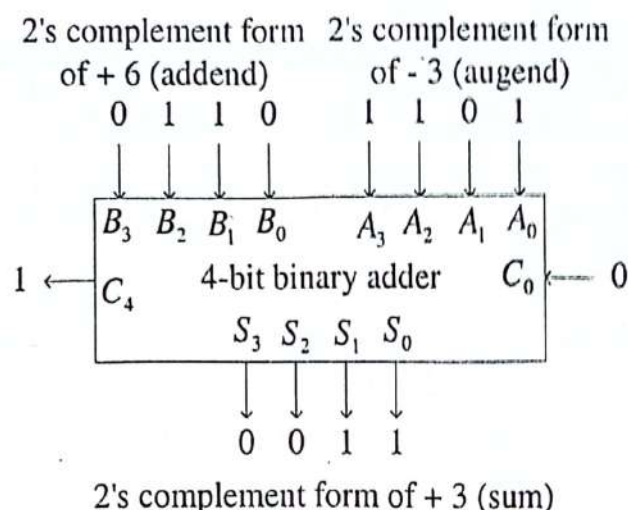


Figure 7.18 Addition of negative and positive number in 2's complement system using binary adder.

7.6.2 2's Complement Subtractor

When the 2's complement system is used, the subtrahend is changed to its 2's complement and then added to the minuend. The sum then represents the difference between the minuend and subtrahend.

The binary adder can be adapted to perform the subtraction of 2's complement signed number if we provide a means for taking the 2's complement of the subtrahend B . The 2's complement of a binary number is obtained by complementing each bit and then adding 1 to the LSB. Figure 7.19 shows how this can be accomplished. The EXOR gates connected to the inputs B_3 through B_0 of the binary adder produce the complement of each bit of subtrahend B . The value of C_0 is 1, so it adds an extra 1 into the LSB of the adder; this forms the 2's complement of B .

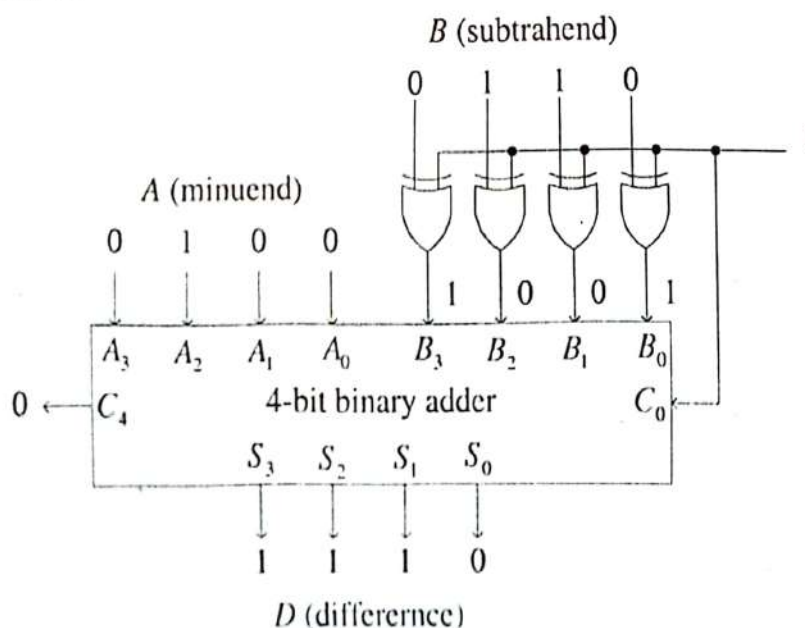


Figure 7.19 4-bit 2's complement subtractor using 4-bit binary adder.

The outputs S_3 through S_0 represent the results of the subtraction operation in 2's complement form. Of course S_3 is the sign bit of the result and indicates whether the result is + or -. The carry output C_4 is disregarded.

To help clarify this operation, subtracting +6 from +4 is shown in Figure 7.19. +4 is applied to the inputs A_3 through A_0 as 0100. +6 is applied as 0110, which is complemented by EXOR gates and applied to the inputs B_3 through B_0 as 1001. The 1001 is added to 0100 by the binary adder along with a 1 added to the LSB position. This produces sum output bits 1110 and a $C_4 = 0$, which is disregarded. This 1110 represents the required difference. Since the sign bit is 1, it is a negative number and is in 2's complement form. The 1110 represents -2 in 2's complement system.

An n -bit 2's complement subtractor can be implemented with n -bit binary adder in a similar manner.

7.6.3 Combined 2's Complement Adder/Subtractor

It should now be clear that the basic binary adder can be used to perform addition or subtraction depending on whether the B number is left unchanged or is converted to 2's complement. A complete circuit that can perform both addition and subtraction in the 2's complement system is shown in Figure 7.20.

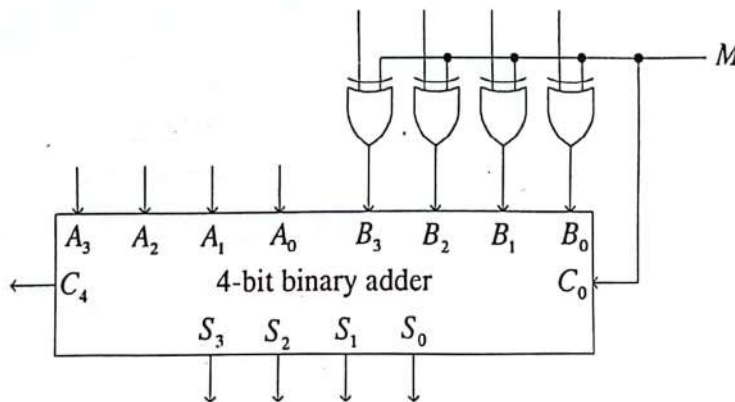


Figure 7.20 4-bit 2's complement adder/subtractor using 4-bit binary adder.

This adder/subtractor circuit is controlled by a mode select input M . When the M input is 0, the EXOR gates do not change the B input and a 0 is applied to the carry input C_0 . So, the circuit performs addition $A + B$. When the M input is 1, the circuit performs subtraction as in Figure 7.19. This type of adder/subtractor circuits are used in computers because they provide a relatively simple means for adding and subtracting signed binary numbers.

7.7 BCD Arithmetic Circuits

Computers or calculators that perform arithmetic operations directly in the decimal number system represent decimal numbers in BCD form. An arithmetic circuit for such a computer must accept BCD numbers and present results in the BCD number. For binary arithmetic, it was sufficient to consider a pair of significant bits at a time,

together with a previous carry or borrow. But a BCD arithmetic circuit requires nine inputs and five outputs, since four bits are required to code each decimal digit and the circuit must have an input carry or borrow and an output carry or borrow. The design of a nine-input, five-output combinational circuit by the classical method requires a truth table with $2^9 = 512$ entries. Many of the input combinations are don't care conditions, since BCD code input has six combinations that are invalid. The simplified Boolean functions for the circuit may be obtained by a computer-generated Quine-McCluskey method, and the result would probably be a connection of gates forming an irregular pattern. An alternate procedure is to add or subtract the numbers with binary adder circuits, taking into consideration the fact that six combinations in each 4-bit input are not used. The output must be modified so that only those binary combinations that are valid combinations of the BCD are generated.

7.7.1 BCD Adder

Consider the arithmetic addition of two decimal digits in BCD, together with a possible carry from a previous stage. Since each input digit does not exceed 9, the output sum cannot be greater than $9 + 9 + 1 = 19$, the 1 in the sum being an input carry. Suppose we apply two BCD digits to a 4-bit binary adder. The adder will form the sum in *binary* and produce a result that may range from 0 to 19. These binary numbers are listed in Table 7.1 and are labeled by symbols K , Z_3 , Z_2 , Z_1 , and Z_0 . K is the carry; Z_3 , Z_2 , Z_1 , and Z_0 are sum bits. The first column in the table lists the binary sums as they appear in the outputs of a 4-bit *binary* adder. The output sum of two *decimal digits* must be represented in BCD and should appear in the form listed in the second column of the table. The problem is to find a simple rule by which the binary number in the first column can be converted to the correct BCD-digit representation of the number in the second column.

In examining the contents of the table, it is apparent that when the binary sum is equal to or less than 1001, the corresponding BCD number is identical, and therefore no conversion is needed. When the binary sum is greater than 1001, we obtain an invalid BCD representation. The addition of binary 6 (0110) to the binary sum converts it to the correct BCD representation and also produces an output carry as required.

The logic circuit that detects the necessary correction can be derived from the table entries. It is obvious that a correction is needed when the binary sum has an output carry $K = 1$. The other six combinations from 1010 to 1111 that need a correction have a 1 in position Z_3 . To distinguish them from binary 1000 and 1001, which also have a 1 in position Z_3 , we specify further that either Z_2 or Z_1 must have a 1. The condition for a correction and an output carry can be expressed by the Boolean function

$$C = K + Z_3Z_2 + Z_3Z_1$$

When $C = 1$, it is necessary to add 0110 to the binary sum and provide an output carry for the next stage.

Table 7.1 Derivation of BCD adder.

Binary sum $K \quad Z_3 \quad Z_2 \quad Z_1 \quad Z_0$					BCD sum $C \quad S_3 \quad S_2 \quad S_1 \quad S_0$					Decimal
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

A BCD adder is a circuit that adds two BCD digits in parallel and produces a sum digit also BCD. The block diagram of a BCD adder is shown in Figure 7.21(a). A BCD adder must include the correction logic in its internal construction. To add 0110 to the binary sum, we use a second 4-bit binary adder as shown in Figure 7.21(b). The two decimal digits, together with the input carry, are first added in the top 4-bit binary adder to produce the binary sum. When the output carry is equal to 0, nothing is added to the binary sum. When it is equal to 1, binary 0110 is added to the binary sum through the bottom 4-bit binary adder. The output carry generated from the bottom binary adder can be ignored, since it supplies information already available at the output-carry terminal.

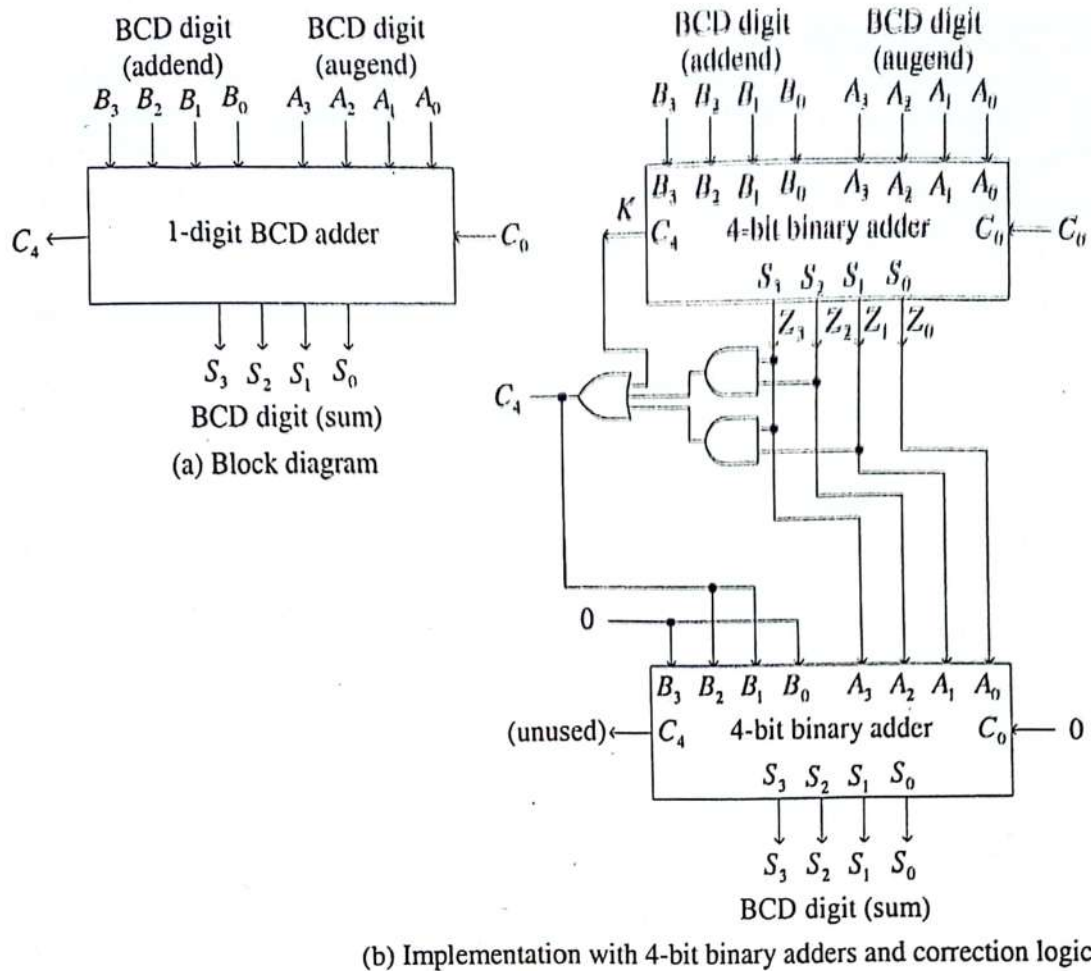


Figure 7.21 1-digit BCD adder.

An n -digit BCD adder needs n cascaded BCD adder stages. The output carry from one stage must be connected to the input carry of the next higher-order stage. Cascading of two BCD adders is shown in Figure 7.22.

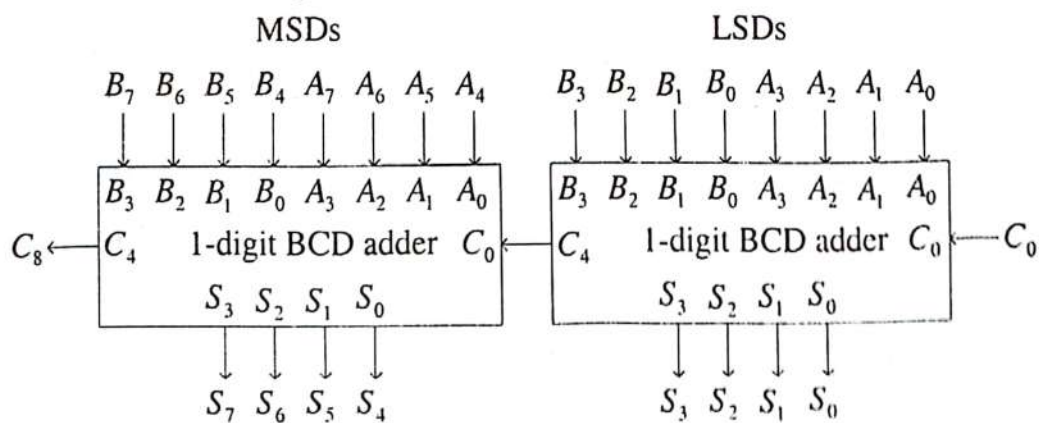


Figure 7.22 Cascading of two 1-digit BCD adders to implement a 2-digit BCD adder.

7.7.2 BCD Subtractor

For BCD subtraction, 9's complement of the subtrahend is added to the minuend. The 9's complement of a BCD number is given by nine minus that number. For example, 9's complement of 7 is $9 - 7 = 2$. In BCD code we say that the 9's complement of 0111 is 0010. Examples of BCD subtraction using 9's complement to represent negative numbers are given below:

(a) Subtract 5 from 9.

$$\begin{array}{r}
 9 = \quad \quad 1001 \\
 -5 = \quad \quad 0100 \text{ (9's complement of 5)} \\
 \hline
 \quad \quad 10011 \text{ (result of BCD addition)} \\
 \quad \quad \quad 1 \text{ add end around carry (EAC)} \\
 \hline
 \quad \quad 0100 = +4
 \end{array}$$

(b) Subtract 8 from 4

$$\begin{array}{r}
 4 = 0100 \\
 -8 = 0001 \text{ (9's complement of 8)} \\
 \hline
 \quad \quad 0101 \text{ (result of BCD addition)}
 \end{array}$$

9's complement of 0101 or 5 is 4
Therefore, the answer is -4

From the above examples, we conclude the following:

1. If the BCD sum of minuend and the 9's complement of subtrahend produces an end around carry (EAC) (example of (a)), then add EAC to the BCD sum. The result is positive number represented by this sum.
2. If the BCD sum of the minuend and the 9's complement of the subtrahend does not produce any EAC, then the result is negative and is in the 9's complement form (example of (b)).

The 9's complement of a BCD digit can be determined by adding 1010 to the 1's complement of the digit. Figure 7.23 shows a 1-digit BCD subtractor. The top 4-bit binary adder produces 9's complement of the subtrahend B by complementing the bits by EXOR gates and then adding 1010. The BCD adder of the middle add the minuend A and 9's complement of the subtrahend B . If the end around carry (EAC) is 1, then the bottom 4-bit binary adder adds 1 with the sum produced by the BCD adder, which gives the difference BCD digit. The sign bit is 0 and the result is positive. If the EAC is 0, then the bottom 4-bit binary adder produces the 9's complement of the sum produced by the BCD adder. The sign bit is 1 and the result is negative.

An n -digit BCD subtractor can be implemented using one n -digit BCD adder and two $4n$ -bit binary adders in a similar manner.

7.7.3 Combined BCD Adder/Subtractor

A complete circuit that can perform both BCD addition and subtraction is shown in Figure 7.24. This BCD adder/subtractor circuit is controlled by a mode selector input M . When the M input is 0, the top 4-bit binary adder adds 0 with B , that is, it passes B unchanged. The middle BCD adder add A and B . The bottom 4-bit adder adds 0 with the sum produced by the middle BCD adder. Therefore, the total circuit

produces $A + B$. When the M input is 1, the total circuit produces $A - B$ as in Figure 7.23.

We observe that the BCD arithmetic is rather complex in comparison to the straight binary arithmetic. Therefore, BCD arithmetic requires more hardware and results in reduction of speed. Also, more number of bits is required to represent a given number in BCD. Therefore, BCD is normally not used in computers. However, digital calculators use BCD because the input data from the keyboard as well as the output display are decimal.

7.8 Available MSI Arithmetic ICs

7.8.1 Look-Ahead Carry Generator

A typical look-ahead carry generator is the IC type 74182. The pin assignment of the IC is shown in Figure 7.25. The IC 74182 accepts four active-LOW Carry Propagate (P'_0, P'_1, P'_2, P'_3) signals, four active-LOW Carry Generate (G'_0, G'_1, G'_2, G'_3) signals, and an active-HIGH Carry Input (C_0). It provides anticipated active-HIGH carries (C_1, C_2, C_3). It also provides active-LOW Carry Propagate (P') and Carry Generate (G') outputs, which may be used for generating C_4 .

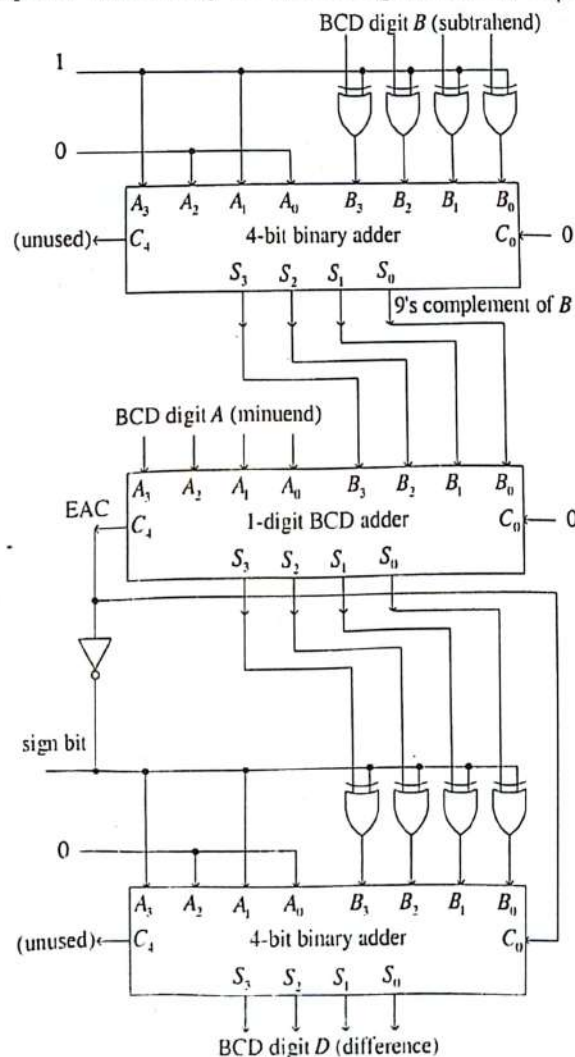


Figure 7.23 1-digit BCD subtractor.

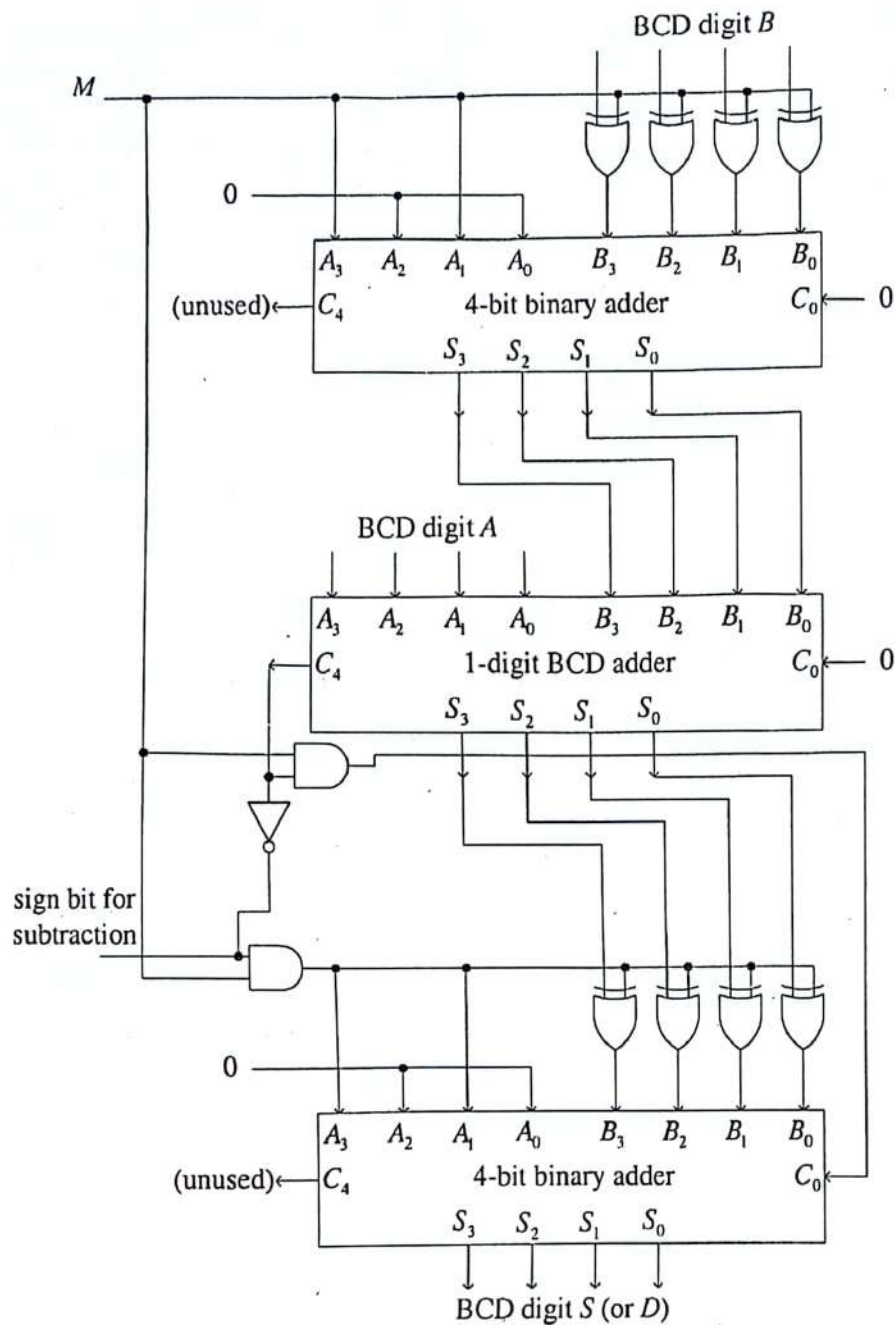


Figure 7.24 1-digit BCD adder/subtractor.

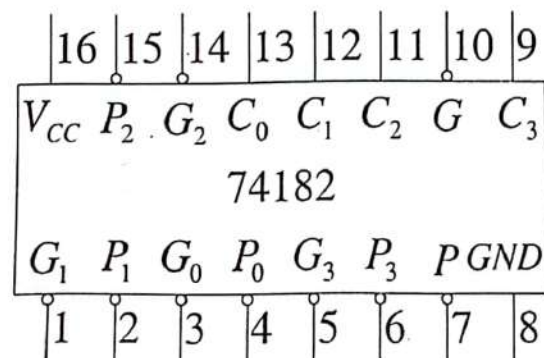


Figure 9.25 IC type 74182 look-ahead carry generator.

The logic equations provided at the outputs are

$$\begin{aligned}C_1 &= G_0 + P_0 C_0 \\C_2 &= G_1 + P_1 G_0 + P_1 P_0 C_0 \\C_3 &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \\G' &= (G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0)' \\P' &= (P_3 P_2 P_1 P_0)'\end{aligned}$$

C_4 can be generated as follows

$$\begin{aligned}C_4 &= G + PC_0 \\&= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0\end{aligned}$$

7.8.2 4-bit Binary Adder

4-bit binary adder is available as MSI IC package. Available 4-bit binary adder ICs are given in Table 7.2.

Table 7.2 Available MSI binary adder ICs.

IC No.	Description
7483	4-bit binary adder
74283	4-bit binary adder with fast carry

TTL IC type 7483 is discussed here. The internal construction of IC type 7483 is similar to Figure 7.17. The pin assignment of the IC is shown in Figure 7.26. The two 4-bit input binary numbers are A_0 through A_3 and B_0 through B_3 . The 4-bit sum is obtained from S_0 through S_3 . C_0 is the input carry and C_4 the output carry.

16	15	14	13	12	11	10	9
B_3	S_3	C_4	C_0	GND	B_0	A_0	S_0
7483							
A_3	S_2	A_2	B_2	V_{CC}	S_1	B_1	A_1
1	2	3	4	5	6	7	8

Figure 7.26 IC type 7483 4-bit binary adder.

7.8 Magnitude Comparator

The comparison of two numbers is an operation that determines if one number is greater than, less than, or equal to the other number. A *magnitude comparator* is a combinational circuit that compares two numbers, A and B , and determines their relative magnitudes. The outcome of the comparison is specified by three binary variables that indicate whether $A > B$, $A = B$, or $A < B$.

The circuit for comparing two n -bit numbers has 2^{2n} entries in the truth table and becomes too cumbersome even with $n = 3$. On the other hand, as one may suspect, a comparator circuit possesses a certain amount of regularity. Digital functions which possess an inherent well-defined regularity can usually be designed by means of an algorithmic procedure if one is found to exist. An *algorithm* is a procedure that specifies a finite set of steps that, if followed, gives the solution to a problem. We illustrate this method here by deriving an algorithm for the design of a 4-bit magnitude comparator.

The algorithm is a direct application of the procedure a person uses to compare the relative magnitude of two numbers. Consider two numbers, A and B , with four digits each. Write the coefficients of the numbers with descending significance as follows:

$$A = A_3A_2A_1A_0$$

$$B = B_3B_2B_1B_0$$

where each subscripted letter represents one of the digits in the number. The two numbers are equal if all pairs of significant digits are equal, i.e., if $A_3 = B_3$ and $A_2 = B_2$ and $A_1 = B_1$ and $A_0 = B_0$. When the numbers are binary, the digits are either 1 or 0 and the equality relation of each pair of bits can be expressed logically with an EXNOR or equivalence function:

$$x_i = A_iB_i + A'_iB'_i \quad i = 0,1,2,3$$

where $x_i = 1$ only if the pair of bits in position i are equal, i.e., if both are 1s or both are 0s. The equality of the two numbers, A and B , is displayed in a combinational circuit by an output binary variable which we designate by the symbol $(A = B)$. This binary variable is equal to 1 if the input numbers, A and B , are equal, and it is equal to 0 otherwise. For the equality to exist, all x_i variables must be equal to 1. This dictates an AND operation of all variables:

$$(A = B) = x_3x_2x_1x_0$$

The binary variable $(A = B)$ is equal to 1 only if all pairs of bits of the two numbers are equal.

To determine if A is greater than or less than B , we inspect the relative magnitudes of pairs of significant bits starting from the most significant position. If the two bits are equal, we compare the next lower significant pair of bits. This comparison continues until a pair of unequal bits is reached. If the corresponding bit of A is 1 and that of B is 0, we conclude that $A > B$. If the corresponding bit of A is 0 and that of B is 1, we have that $A < B$. The sequential comparison can be expressed logically by the following two Boolean functions:

$$(A > B) = A_3B'_3 + x_3A_2B'_2 + x_3x_2A_1B'_1 + x_3x_2x_1A_0B'_0$$

$$(A < B) = A'_3B_3 + x_3A'_2B_2 + x_3x_2A'_1B_1 + x_3x_2x_1A'_0B_0$$

The symbols $(A > B)$ and $(A < B)$ are *binary* output variables which are equal to 1 when $A > B$ or $A < B$, respectively.

The gate implementation of the three output variables just derived is simpler than it seems because it involves a certain amount of repetition. The “unequal” outputs can use the same gates that are needed to generate the “equal” output. The logic diagram of the 4-bit magnitude comparator is shown in Figure 7.25. This is a multilevel implementation and, as clearly seen, it has a regular pattern. The procedure for obtaining magnitude comparator circuits for binary numbers with more than four bits should be obvious from this example. The same circuit can be used to compare the relative magnitudes of two BCD digits.

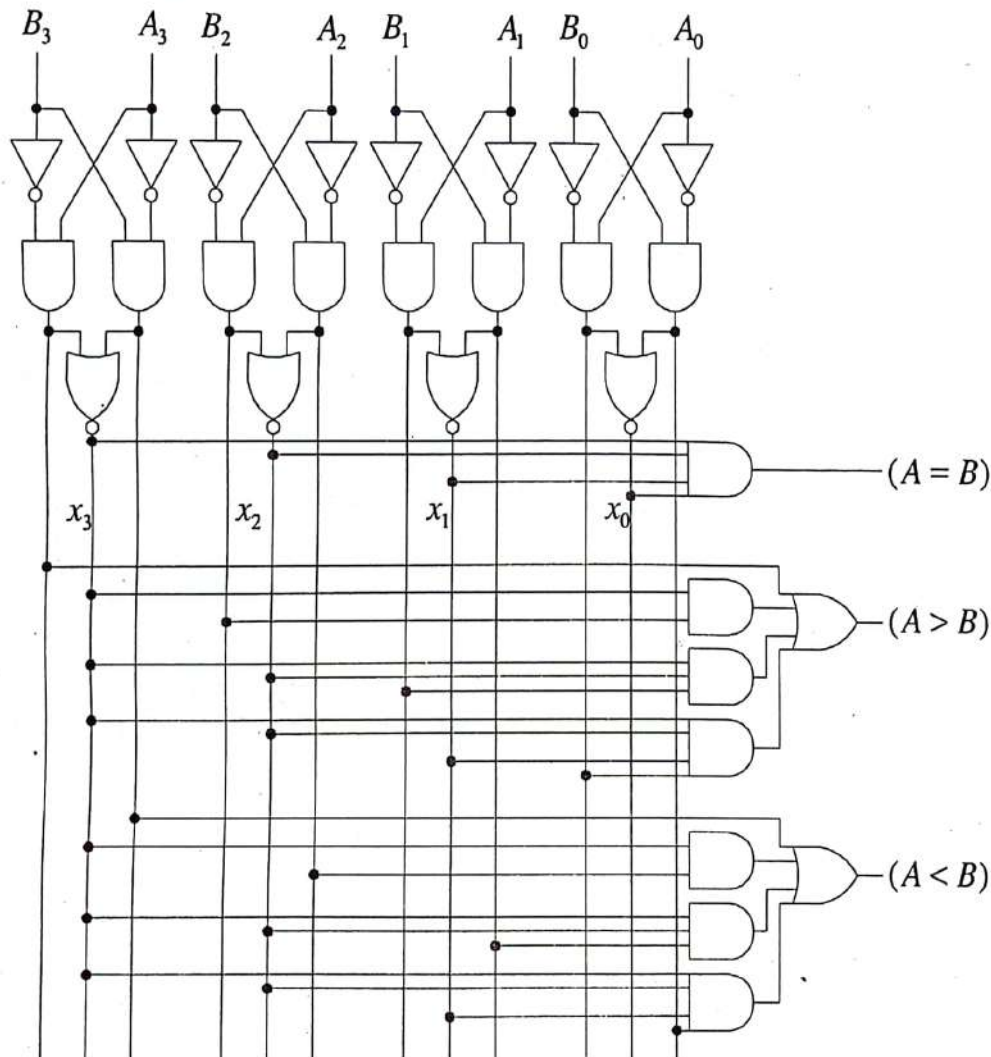


Figure 7.25 4-bit magnitude comparator.

Exercises

- 7.1 Implement a full-subtractor with two half-subtractors and an OR gate.
- 7.2 Show how a full-adder can be converted to a full-subtractor with the addition of one NOT gate.
- 7.3 Design a BCD-to-excess-3 code converter using a 4-bit binary adder.
- 7.4 Design an excess-3-to-BCD code converted using a 4-bit binary adder.
- 7.5 Using four 4-bit binary adders, construct a 16-bit binary adder.
- 7.6 Design a 4-bit BCD adder using 4-bit binary adders.
- 7.7 Design a circuit that compares two 4-bit numbers, A and B , to check if they are equal. The circuit has one output x , so that $x = 1$ if $A = B$, and $x = 0$ if $A \neq B$.

Further Reading

1. *The TTL Data Book*. Dallas, Texas Instruments, 1988.
2. Richard S. Sandige, *Modern Digital Design*, McGraw-Hill Publishing Company, 1990.
3. R. P. Jain, *Modern Digital Electronics*, 2nd ed., Tata McGraw-Hill Publishing Company Limited, India, 1998.
4. *Microprocessor Data Hand Book*, Revised Ed. BPB Publications, India, 2000.
5. M. Morris Mano, *Digital Logic and Computer Design*, Prentice-Hall, Inc., 2000.
6. John P. Uyemura, *Digital Systems Design: An Integrated Approach*, Brooks/Cole Publishing Company, 2000.
7. D. C. Green, *Digital Electronics*, 5th ed., Pearson Education, 2001.
8. Ronald J. Tocci and Neal S. Widmer, *Digital Systems: Principles and Applications*, 8th ed., Prentice-Hall, Inc., 2001.
9. V. K. Jain, *Switching Theory and Digital Electronics*, Khanna Publishers, India, 2002.
10. M. Morris Mano, *Digital Design*, 3rd ed., Pearson Education, 2002.
11. Samuel C. Lee, *Digital Circuits and Logic Design*, Prentice-Hall, Inc., 2002.